Modelling Dual-Deployment Rocket Recovery Systems Under Constant Wind Conditions

Semester: Winter 2025

Name: Rhea Zhu

Table of Contents

Modelling Dual Deployment Rocket Recovery Systems Under Constant Wind Conditions

1. Overview

In this project, I use Euler's method to investigate the conditions in a dual-deployment parachute system needed to safely land a 70kg rocket launched with 3000N of force over 5 seconds. Specifically, I fix properties on the drag parachute and vary (1) the coefficient of drag and (2) surface area of the main parachute, as well as (3) the launch angle of the parachute. Altogther, I simulate two things:

1. The force the rocket experiences when the main parachute is deployed, and
2. The force the rocket experiences when it collides with the ground.

We define the coordinate system of the rocket's trajectory as follows:


Small

Where the rocket ascends in the positive y-direction, and the launch angle θ is the angle made with the positive x-axis. Furthermore, the recovery trajectory occurs under a constant wind of $3\mathrm{m}/\mathrm{s}$ in the positive x direction; this wind speed was chosen because it is roughly the average wind speed in Vancouver from October to April [15]. The rocket recovery process is considered a success if

1. The forces in both parts (1) and (2) are less than or equal to 3500 Newtons.
2. The rocket lands 100 metres (horizontally) from its original position.

Research Question: Given the initial deployment conditions of a drag parachute, what cross-sectional area and drag coefficient of the main parachute, as well as the rocket launch angle, in a dual deployment system are needed to create a successful landing?

1.1 Introduction

1.1.1 What is Rocket Recovery?

In rocketry, a significant portion of flight occurs after a rocket reaches its apogee (peak height) and begins returning to land. During this trajectory, the rocket approaches the ground at high velocities, accelerating towards Earth. Due to the drastic forces and high velocities, rockets can easily be damaged while airborne and upon colliding with the ground. Rocketry recovery refers to the systems and tools used to minimize such risks; this includes developing simulations to test the effectiveness of various recovery techniques. An ideal recovery should both land the rocket at low velocities and at the same place over multiple launches. [1][2][3]

1.1.2 Parachutes in Rocket Recovery

A parachute (also known simply as a “chute”) is designed to slow the descent of objects in the air and is often used in rocket recovery. When accelerating towards the ground, the parachute is deployed from a storage bay in the rocket, inflating into a large canopy; this canopy envelops air under itself, creating an "opposing force" of air resistance (also known as drag force) to counteract the downwards gravitational force. This slows the rocket down as it approaches Earth, decreasing its impact with the ground. While larger parachutes typically create more air resistance, they also produce more drift, causing landings to be imprecise. [3] [4]

1.1.3 Dual Deployment Parachute Systems

A common parachute system in rocketry is the dual-deployment system, where two parachutes are released at different altitudes during recovery. The first parachute, known as the **"drag chute,"** is typically smaller and is deployed to slightly slow the rocket's descent while not creating excess drift. The second parachute, known as the **"main chute,"** is deployed later; this parachute is significantly larger, creating more drag and further decreasing the rocket's velocity to produce a safe, gentle landing. [4][5]

The benefits of a parachute dual-deployment are two-fold. Firstly, as alluded to before, this system decreases the drift the rocket may experience; by deploying a smaller parachute first, less air is captured underneath the canopy, preventing drastic horizontal forces pushing the rocket to veer off-course. More importantly, though, a dual-deployment system reduces external forces on the rocket while airborne. If the large main parachute were immediately deployed during its downward trajectory, the drastic change in momentum over a short period would create a large, sudden force on the rocket, leading to rocket damage. By first deploying a smaller drogue parachute, the gradual change in velocity minimizes this possibility, ultimately increasing the chance of rocket recovery success. [4][5]

1.2 Experimental Method & Analysis

The purpose of this project is to find the launch angle of the rocket, coefficients of drag, and surface area in the main parachute needed to create a successful rocket landing, given initial rocket and drogue parachute conditions. In this experiment, we define a “successful rocket landing” as the following:

1. **Collision 1 (Air)**: The force experienced by the rocket between the drogue and main parachute deployment is under 3500 N.
2. **Collision 2 (Ground)**: The force experienced by the rocket during its collision with the ground is under 3500 N.
3. **Landing**: Under constant wind conditions of $3\mathrm{m}/\mathrm{s}$, the rocket must land 100m within its starting position.

The threshold force of 3500 N was approximated by online estimates of 5G's of force on a 70kg rocket [6].

1.2.1 Simulation Setup

Part 0: A 70 kg rocket is launched at an angle θ with 3000N of thrust force over a period of 5 seconds; these values were estimated from UBC Rocket's past projects, multiple IREC launching videos, and the impulse of a Class-N motor [7][8][9][10]. The angle θ is defined as the angle the rocket makes with the positive x-axis. During its ascent, the rocket is assumed to not experience any drag.

Part 0.5, Parachute 1 Deployment: When the rocket has reached its apogee, a drogue parachute with a 0.7 coefficient of drag and a cross-sectional surface area of $5m^2$ is deployed, and the rocket begins its descent. In this simulation, we assume that Parachute 1 is instantaneously deployed (see justification in *Physical Assumptions*). Parachute 1's coefficient of drag and cross-sectional surface area were determined from various parachute testing and manuals found online [11][12].

Part I, Parachute 2 Deployment: When the rocket has descended to an altitude of 500m, a main parachute with a variable coefficient of drag and cross-sectional area is deployed over 0.3s; this period of time is estimated from the Orion Parachute Test in 2014 [13]. The force experienced by the rocket during this deployment is calculated. If the force exceeds 3500N, the simulation ends and displays velocity vs. time and position vs. time graphs. If not, the simulation continues to Part II.

After the main parachute is deployed, the drogue parachute is detached from the rocket. Since the main parachute is significantly larger and has a higher coefficient of drag, this chute's detachment does not have a significant impact on the net drag force. Therefore, we assume that the Parachute 1 detachment does not affect the rocket's trajectory.

Part II, Rocket Descent: The rocket descends until it reaches the ground.

Part III, Ground Collision: The rocket collides with the concrete ground over the course of 0.1 seconds. This time period was approximated from papers detailing drop tests of rigid objects on concrete [14], fully decelerating until rest. The force experienced by the rocket during the ground collision is calculated. If the force exceeds 3500N, the simulation indicates a failure in rocket landing. If not, the rocket landing has succeeded

Part IV, Post Ground Collision: After the ground collision, the rocket remains at rest on the ground; depending on whether its ground collision force exceeds the threshold force, it is either damaged (recovery fail) or not (recovery success). This ends the simulation

Throughout the rocket's trajectory there is a constant wind speed of $3\mathrm{m}/\mathrm{s}$ to the right.

1.2.2 Programming

Part 0: Euler's method is used to simulate the rocket's ascent. The rocket's individual vertical and horizontal components of velocity are calculated based on the fixed angle of the rocket launch. We assume that the surface area the rocket body makes with the air is minimal, meaning that there is no drag or wind effects.

Part 0.5: After parachute 1 is deployed, an additional drag force is introduced into the simulation, where

$$F_D = \frac{1}{2} C_D \rho A v^2$$

where C_D is the parachute's coefficient of drag, ρ is air density, A is the parachute's cross-sectional surface area in contact with air, and v is rocket velocity. Since the drag force changes with velocity, the use of Euler's method is adapted from Part 0 to account for the changing force.

The account for wind, the velocity used in calculating total and component drag force is the **relative velocity** of the rocket with the air. As a result, the $3\mathrm{m}/\mathrm{s}$ is subtracted from the horizontal component of the rocket's velocity relative to the ground to find total drag. The horizontal and vertical components of drag are then calaculated using the angle made by the velocities of the rocket relative to the air.

Part I: Given the chosen coefficient of drag and surface area, the terminal velocity of parachute 2 is calculated. Since this velocity will be much less than the velocity of the rocket before chute 2 deployment, there will be a drastic change in momentum, resulting in a large force:

$$F = \frac{\Delta p}{\Delta t} = \frac{m \times \Delta v}{\Delta t} = \frac{m \times (v_i - v_{term})}{0.3}$$

If the force exceeds 3500N, the simulation ends and displays velocity vs. time and position vs. time graphs. If not, the simulation continues to Part II.

Part II, Rocket Descent: Euler's method is used to simulate the rocket descent, where its horizontal velocity component is the wind's velocity, and the vertical velocity component is the calculated vertical terminal velocity.

Part III, Ground Collision: During ground collision, the rocket loses all its momentum in 0.1s. This creates a massive force on the rocket:

$$F = \frac{\Delta p}{\Delta t} = \frac{m \times \Delta v}{\Delta t} = \frac{m \times v_{term}}{0.1}$$

If the force exceeds 3500N, the simulation indicates a failure in rocket landing. If not, the rocket landing has succeeded.

Part IV, Post Ground Collision: No analysis is conducted for Part IV, since the rocket has fully decelerated and remains at rest on the ground.

1.2.3 Analysis Method

The analysis of this project will be separated into three parts; each analysis portion builds off of the previous one.

1. **Preliminary Trials Without Wind**: In this first section, we investigate the rocket's trajectory and velocity first without wind and variable launch angle in a one-dimensional space, where the rocket is launched upwards ($\theta = 90^o$). We do this without wind first, to gauge a preliminary understanding of how the rocket would ideally behave in terms of its terminal velocity and parachute deployments. Since this portion will not be used in our final two-dimensional analysis, we pick the thrust force to be 1000N instead of 3000N. This is chosen to see that behaviour of the rocket more clearly; at 3000N, we find that the rocket reaches terminal velocity too quickly for the graph to properly display its trajectory before parachute 2's deployment. We analyse this through three different combinations of coefficient of drag and surface area.
2. **Preliminary Trials With Wind**: Next, we extend Part I by introducing wind and launch angle as additional simulation components and observe how they affect the rocket system. From this, we make preliminary conclusions how launch angle, coefficient of drag, and surface area components affect the rocket's trajectory.
3. **Quantitative Phase Space Scan**: We end our analysis by conducting a quantitative phase space scan of a wide range of launch angles, coefficient of drag, and surface areas. Using this, we can find what combinations of these three parameters lead to a successful rocket recovery.

1.3 Physical Assumptions

In my project, I make the following assumptions:

- **Environment**: Air density is constant at $1.225 kg/m^3$.
- **Energy Loss**: There is no energy loss other than air resistance in the system.
- **Launch**: The rocket's trajectory is only along the x-y plane; there is no z axis component.
- **Parachute Properties**:
    1. Parachute 1 is instantaneously deployed. Since it occurs when the velocity is 0, the force acting on the rocket will be 0N anyway; therefore, a non-zero time deployment would not affect the simulation's calculations. We simplify the model to assume instantaneous deployment without compromising the integrity of the simulation.
    2. The parachutes are massless.
    3. Parachute 1 detaches right after Parachute 2 is deployed, and does not affect the rocket's aerodynamics.
    4. Parachute 2's deployment takes 0.3 seconds.
- **Rocket Properties**: We assume that the rocket is point-like and has no properties other than mass that would affect its aerodynamics and collisions.
- **Ground Collision**: We assume that it takes 0.1s for the rocket and ground to collide. We also assume that the ground collision is fully inelastic; this assumption is reasonable, since rockets typically lose all gravitational potential and kinetic energy upon landing.

1.3.1 Consequences of Physical Assumptions

The above physical assumptions and neglected effects may influence the external validity of simulation results, since it doesn't perfectly model the trajectory of a dual-deployment rocket recovery in real life. For instance, chute detachment and air density variations would cause turbulence on the rocket, creating components of velocities not accountd for during rocket dscent. Furthermore, the rocket body's dimensions would also affect its descent due to the center of mass, excess drag, and other complex aerodynamics, impacting the rocket's descent time, velocity, and horizontal displacement upon landing. We ignore these effects in this simulation to maintain simplicity.

2. Analysis: Investigating Forces and Position in a Dual-Deployment Parachute System

2.1 Preliminary Trials Without Wind

We first observe the effects of different coefficients of drag and surface area on recovery success through three attempts.

Attempt 1: Coefficient of Drag = 1.5, Surface Area = 130 m^2

We analyze and plot Velocity vs. Time and Position vs. Time graphs when the $C_D = 1.5$ and $A = 130m^2$.

In [27]:
```
#====================START SIMULATION CODE====================

import numpy as np
import matplotlib.pyplot as plt

## USER INPUTS_THRESHOLD FORCE AND THRUST FORCE

thrust_force = 1000
thresh_force = 3500

#==================== LAUNCHING  ====================

## Creating empty lists and initial values for each variable
mass = 70 # Rocket kg
force_launch = []

height_launch = []
height_launch.append(0)

acceleration_launch = []

velocity_launch = []
velocity_launch.append(0)

t_launch = []
t_launch.append(0)
dt_launch = 0.1


##########
```

Code Summary: This code block creates empty lists and initial values for the needed variables for the simulation, including force, height, acceleration, velocity, and time. The rocket's thrust force and threshold force of 3500 are also defined.

In [28]:
```
#===============PART 0: ASCENT TO APOGEE ===============

#### Modeling ascent using Euler's Method with while loop

j = 0
while True:
    if t_launch[j] <= 5:
        force_launch.append(thrust_force - mass * 9.8)
    else:
        force_launch.append(-mass*9.8)
    acc = force_launch[j]/mass
    acceleration_launch.append(acc)
```

```
    velocity_launch.append(velocity_launch[j] + acceleration_launch[j])
    height_launch.append(height_launch[j] + velocity_launch[j] * dt_launch)
    t_launch.append(t_launch[j] + dt_launch)

    j = j + 1
    if velocity_launch[j] <= 0: # If reaches deployment height
        break
```

Code Summary: This code block models the rocket's ascent using Euler's method. When the time from launch is under 5 seconds, the rocket experiences both gravity and thrust force. However, after that, the rocket only experiences gravity. Each time step's acceleration, velocity, and height are calculated from force, and a timestep is added after each loop. The ascent stops once the rocket's velocity becomes zero or negative, indicating that the rocket is at its apogee.

In [29]:
```
#==================== RECOVERY ====================

## Set initial variables

t0 = 0      # starting time, s
dt = 0.05   # time step, s
height = height_launch[j] #apogee height, m
dheight = 100 #deployment height, m
rho = 1.225 #air density, kg/m^3
deploy_time = 0.3 #parachute 2 deployment period
collide_time = 0.1 #rocket collision time, s

t_max = 6000 #maximum time
n_steps = int((t_max - t0) / dt) + 1 # Total number of steps

## Arrays to save simulation information

time = np.zeros(n_steps)
position = np.zeros(n_steps)
velocity = np.zeros(n_steps)
drag = np.zeros(n_steps)
acc = np.zeros(n_steps)
instantforce = np.zeros(n_steps)

# Initial properties: Rocket

y1_initial = height # m
v1_initial = 0  # m/s
acc_initial = 0 #ms^2
force_initial = 0 #N
drag_initial = 0 #N
time[0] = t0 #s

r1 = 0.05 # m
position[0] = y1_initial #m
velocity[0] = v1_initial  #m/s
acc[0] = acc_initial #m/s^2
instantforce[0] = force_initial #Force on rocket at apogee, N
drag[0] = drag_initial # Drag force on rocket at apogee, N

gforce = mass * 9.81 # Force of gravity

# Fixed Initial properties: Parachute 1

cod1 = 0.7 #coefficient of drag
area1 = 5 #cross-sectional surface area, m^2

# Variable initial properties: Parachute 2

cod2 = 1.5 #coefficient of drag
area2 = 130 #cross-sectional surface area, m^2
termvel = np.sqrt(2*mass*9.81/(rho*area2*cod2))  #terminal velocity, m/s^2
```

Code Summary: This is the beginning of the rocket's recovery trajectory. This block defines all necessary variables, arrays, and constants. This includes Parachute 1's and Parachute 2's coefficients of drag and cross-sectional surface area. In this simulation, the initial properties of Parachute 2 are variable, but Parachute 1 is fixed.

In [30]:
```
#================= PART 0.5: AFTER PARACHUTE 1 DEPLOYED ==================
```

```
counter = 0
for i in range(1,n_steps):
    drag[i] = 0.5 * rho * cod1 * area1 * velocity[i-1]**2
    instantforce[i] = -gforce + drag[i]
    acc[i] = instantforce[i]/mass

    velocity[i] = velocity[i-1] - acc[i] * dt
    position[i] = position[i-1] - velocity[i] * dt

    time[i]  =  time[i-1] + dt
    counter = counter + 1
    if position[i] <= dheight: # If reaches deployment height
        break
```

Code Summary: This code block models the rocket's descent after parachute 1 is deployed at the apogee using Earler's method and a for loop. The loop breaks when the rocket's position is less than or equal to dheight, signaling to the simulation that Parachute 2 should now be deployed.

In [31]:
```
#========== PART I: PARACHUTE 2 DEPLOYMENT ==================

mi = mass * velocity[counter-1]
mf = mass * termvel
dm = abs(mi - mf)
force = dm/deploy_time
```

Code Summary: This block models the rocket's parachute 2 deployment. The initial and final momentum is calculated through mi and mf, and the difference is calculated as dm. Force is then the change in momentum over deployment time, which was defined previously.

In [32]:
```
# Parachute 2 Deployment Failure

if force > thresh_force:
    for i in range(len(velocity)):
        velocity[i] = -velocity[i]
    fig, axes = plt.subplots(1, 2)
    plt.subplots_adjust(wspace=0.5)
    fig.set_size_inches(9, 5)
    axes[0].plot(time[:counter+1],velocity[:counter+1], color = "brown")
    axes[0].set_xlim(0, 100)
    axes[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
    axes[0].set_xlabel("Time (s)")
    axes[0].set_ylabel("Velocity (m/s)")
    axes[0].text(time[counter], velocity[counter], 'Recovery Fail: \n Rocket Damaged', horizontalalignme
        bbox=dict(
            boxstyle='round,pad=0.5',
            fc='red',
            ec='black',
            lw=2,
            alpha=0.4
        ))

    axes[1].plot(time[:counter+1],position[:counter+1], color = "orange")
    axes[1].set_xlim(0,100)
    axes[1].set_title("Rocket Position vs. Time", fontweight = "bold")
    axes[1].set_xlabel("Time (s)")
    axes[1].set_ylabel("Position (m)")
    axes[1].text(time[counter], position[counter], 'Recovery Fail: \n Rocket Damaged', horizontalalignme
        bbox=dict(
            boxstyle='round,pad=0.5',
            fc='red',
            ec='black',
            lw=2,
            alpha=0.4
        ))

    fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
    plt.subplots_adjust(top=0.8)
    fig.text(0.5, 0.88, 'Status: Failed during Parachute 2 Deployment', ha='center', fontsize=14, color=

    print("Collision 1 Failed: Rocket experienced force greater than threshold during Parachute 2 deploy
    print(f"The threshold force was {thresh_force: 0.2f} N.")
    print(f"The force during parachute 2 deployment was {force: 0.2f} N.")
```

```
# CODE SUMMARY [1]

# Parachute 2 deployment success
else:
    print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployme
    dt = 1
    counter2 = counter

#================= PART II: CONSTANT DESCENT TO GROUND ==================

    for i in range(counter,n_steps):
        position[i] = position[i-1] - termvel * dt
        counter2 = counter2 + 1
        time[i]  =  time[i-1] + dt
        velocity[i] = termvel
        if position[i] <= 0:
            velocity[i] = 0
            position[i] = 0
            break

# CODE SUMMARY [2]

# PART III: GROUND COLLISION
    m11 = mass * velocity[counter2-2]
    force2 = m11/collide_time


# CODE SUMMARY [3]

# GROUND COLLISION FAILURE
    if force2 >= thresh_force:
        for i in range(len(velocity)):
            velocity[i] = -velocity[i]

        print("Collision 2 Fail: Rocket experienced force greater than threshold during ground collision
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {force: 0.2f} N.")
        print(f"The force during the ground collision was {force2: 0.2f} N.")

        fig, axes2 = plt.subplots(1, 2)
        plt.subplots_adjust(wspace=0.5)
        fig.set_size_inches(13, 7)

        axes2[0].plot(time[:counter2],velocity[:counter2], color = "brown")
        axes2[0].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
        axes2[0].set_xlabel("Time (s)")
        axes2[0].set_ylabel("Velocity (m/s)")
        axes2[0].text(time[counter-1], (3*velocity[counter-1]-velocity[counter])/4, 'Success: Chute 2 \n
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[0].text(time[counter2-1], velocity[counter2-1], 'Fail: Damaged \n While Landing', horizont
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='red',
                ec='black',
                lw=2,
                alpha=0.4
            ))

        axes2[1].plot(time[:counter2],position[:counter2], color = "orange")
        axes2[1].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[1].set_title("Rocket Position vs. Time", fontweight = "bold")
        axes2[1].set_xlabel("Time (s)")
        axes2[1].set_ylabel("Position (m)")
        axes2[1].text(time[counter-1], (position[0] + position[counter-1])/2, 'Success: Chute \n 2 Deplo
            bbox=dict(
```

```
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[1].text(time[counter2-1], position[counter2-1], 'Fail: Damaged \n While Landing', horizont
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc="red",
                ec='black',
                lw=2,
                alpha=0.4
            ))

        fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
        plt.subplots_adjust(top=0.72)
        fig.text(0.5, 0.89, 'Part I Status: Parachute 2 Deployment - Successs', ha='center', fontsize=14
        fig.text(0.5, 0.85, 'Part II Status: Rocket Reaching Earth - Success', ha='center', fontsize=14,
        fig.text(0.5, 0.81, 'Part III Status: Rocket Landing Safely - Fail', ha='center', fontsize=14, c

# CODE SUMMARY [4]

# GROUND COLLISION SUCCESS, ROCKET RECOVERY SUCCESS
    else:
        for i in range(len(velocity)):
            velocity[i] = -velocity[i]
        print("Collision 2 Success: Rocket experienced force less than threshold during ground collision
        fig, axes2 = plt.subplots(1, 2)
        plt.subplots_adjust(wspace=0.5)
        fig.set_size_inches(12, 6)

        axes2[0].plot(time[:counter2],velocity[:counter2], color = "brown")
        axes2[0].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
        axes2[0].set_xlabel("Time (s)")
        axes2[0].set_ylabel("Velocity (m/s)")
        axes2[0].text(time[counter-1], (3*velocity[counter-1]-velocity[counter])/4, 'Success: Chute 2 \n
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[0].text(time[counter2-1], velocity[counter2-1], 'Success: Landed \n Safely', horizontalali
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))

        axes2[1].plot(time[:counter2],position[:counter2], color = "orange")
        axes2[1].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[1].set_title("Rocket Position vs. Time", fontweight = "bold")
        axes2[1].set_xlabel("Time (s)")
        axes2[1].set_ylabel("Position (m)")
        axes2[1].text(time[(counter-1)]/2,(position[0] + position[counter-1])/2, 'Success: Chute \n 2 De
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[1].text(time[counter2-1], position[counter2-1], 'Success: Landed \n Safely', horizontalali
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc="green",
                ec='black',
                lw=2,
                alpha=0.4
```

```
            ))

        fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
        plt.subplots_adjust(top=0.72)
        fig.text(0.5, 0.89, 'Part I Status: Parachute 2 Deployment - Successs', ha='center', fontsize=14
        fig.text(0.5, 0.85, 'Part II Status: Rocket Reaching Earth - Success', ha='center', fontsize=14,
        fig.text(0.5, 0.81, 'Part III Status: Rocket Landing Safely - Success!', ha='center', fontsize=1
        print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 depl
        print("Collision 2 Success: Rocket experienced less force than threshold during ground collision
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {force: 0.2f} N.")
        print(f"The force during the ground collision was {force2: 0.2f} N.")

trial1vel = velocity.copy()
trial1position = position.copy()
trial1time = time.copy()
trial1count = counter+1

# CODE SUMMARY [5]
```

```
Collision 1 Failed: Rocket experienced force greater than threshold during Parachute 2 deployment
The threshold force was  3500.00 N.
The force during parachute 2 deployment was  3616.64 N.
```

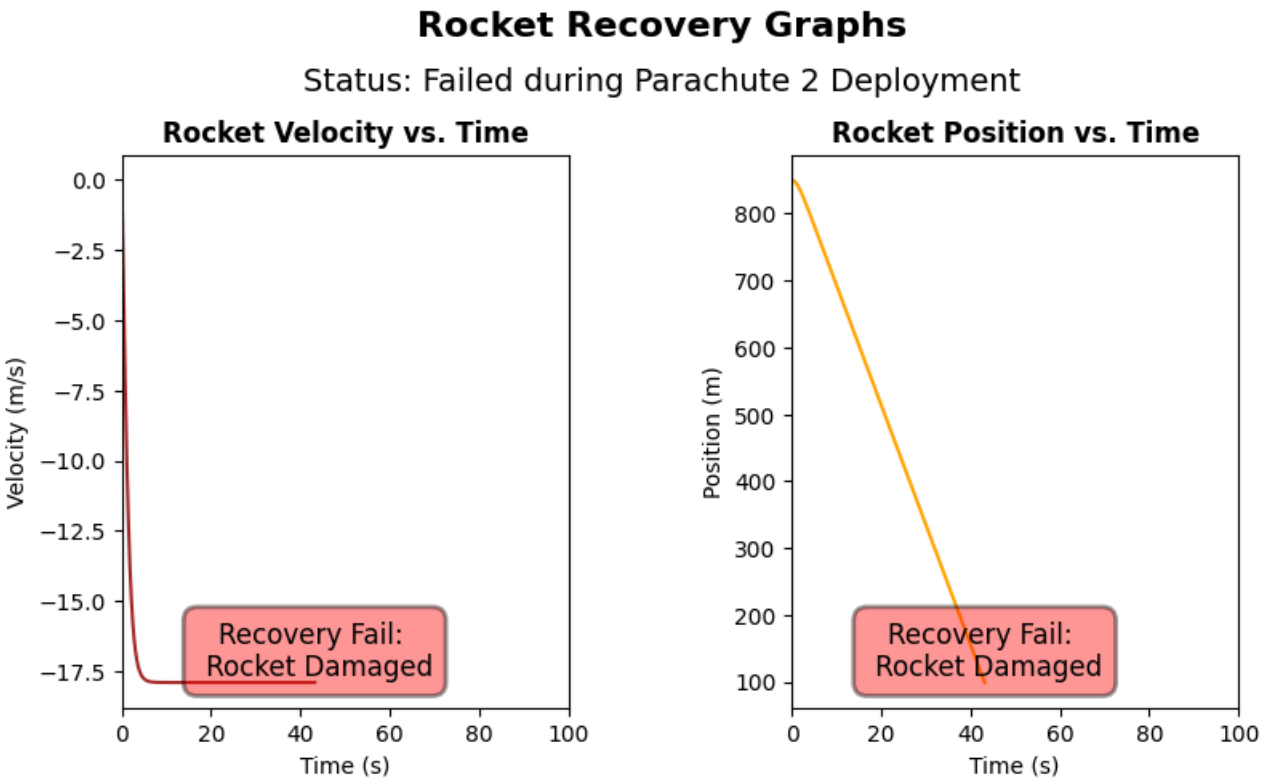


Figure 1. Recovery fail occurs when deploying parachute 2, given Coefficient of Drag = 1.5, Surface Area = 130 m/s^2. The force during parachute 2 deployment exceeds the threshold force. The force during parachute 2 deployment was 3616.64N, while the threshold force was 3500N. This shows that an overly high coefficient of drag and/or surface area results in a parachute 2 deployment fail, due to a large change in momentum.

Note that $t = 0$ corresponds to the apogee (peak height), and that ascent data are not included.

CODE SUMMARIES (See comments in above code)

Code Summary [1]: This code executes the simulation's output if the Parachute 2 deployment force exceeds the threshold force. A graph of the rocket's trajectory and another of its velocity up until Parachute 2 deployment is created, with annotations on the graph indicating a recovery fail. Furthermore, the force during parachute 2 deployment is printed out.

Code Summary [2]: This code simulates the rocket's constant descent to ground given a successful Parachute 2 deployment. Similar to the previous code, it uses Euler's method to continue the rocket's trajectory. A new counter, counter2, is made to extend counter 1; this is used as a tracker across the for loop.

Code Summary [3]: This code simulates the rocket's ground collision upon landing. Since we assume an entirely inelastic collision, the change in momentum is simply the momentum the rocket has immediately before its collision. The force is that momentum divided by its collision time, which was defined in the previous code.

Code Summary [4]: This code executes the simulation's output if the ground collision force exceeds the threshold force. It displays a Rocket Velocity vs. time graph up until the rocket lands, along with annotations displaying a successful Parachute 2 deployment but a failed ground collision. The forces during parachute 2 deployment and ground collision are also printed.

Code Summary [5]: This code executes the simulation's output if the ground collision force is less than the threshold force, creating a successful rocket recovery. It displays a Rocket Velocity vs. Time graph up until the rocket lands, along with annotations displaying a successful Parachute 2 deployment and ground collision. The forces during parachute 2 deployment and ground collision are also printed.

Attempt 1: Analysis

From Rocket Velocity vs. Time graph, the velocity of the rocket initially has a positive slope, whose derivative slowly decreases until it is 0. This is indicative of the rocket reaching the terminal velocity of Parachute 1. In the Rocket Position vs. Time graph, we also see a slight curve at the beginning of the graph, showing the rocket's change in displacement as it approaches terminal velocity. However, since the rocket approaches Parachute 1's terminal velocity so quickly, this nuance in the graph is barely noticeable.

When Parachute 2 is deployed, the massive change in momentum results in a 3617N force on the rocket, exceeding the threshold force by 117N. This damages the rocket, leading to a **recovery failure**.

To improve upon this, we must decrease the coefficient of drag and/or the cross-sectional surface area of Parachute 2. This will increase the terminal velocity of Parachute 2, creating a smaller change in momentum and force between Parachute 1 and Parachute 2 deployment.

Attempt 2: Coefficient of Drag = 1.25, Surface Area = 25 m^2

From Attempt 1, we decrease both the coefficient of drag and cross-sectional surface area of Parachute 2. We analyze and plot Velocity vs. Time and Position vs. Time graphs when the $C_D = 1.25$ and $A = 25m^2$.

In [33]:
```
import numpy as np
import matplotlib.pyplot as plt


def run_simulation(area2, cod2):

    # USER INPUTS_THRESHOLD FORCE AND THRUST FORCE

    thrust_force = 1000
    thresh_force = 3500

    #==============LAUNCHING==================

    # Creating empty lists and initial values for each variable

    mass = 70 # Rocket kg
    force_launch = []

    height_launch = []
    height_launch.append(0)

    acceleration_launch = []

    velocity_launch = []
    velocity_launch.append(0)

    t_launch = []
    t_launch.append(0)
    dt_launch = 0.1

    #==============PART 0: ASCENT TO APOGEE==================
```

```
#### Modeling ascent using Euler's Method with while loop

j = 0
while True:
    if t_launch[j] <= 5:
        force_launch.append(thrust_force - mass * 9.8)
    else:
        force_launch.append(-mass*9.8)
    acc = force_launch[j]/mass
    acceleration_launch.append(acc)
    velocity_launch.append(velocity_launch[j] + acceleration_launch[j])
    height_launch.append(height_launch[j] + velocity_launch[j] * dt_launch)
    t_launch.append(t_launch[j] + dt_launch)

    j = j + 1
    if velocity_launch[j] <= 0: # If reaches deployment height
        break

#==================== RECOVERY ====================

# Set initial variables

t0 = 0      # starting time, s
dt = 0.05   # time step, s
height = height_launch[j] #apogee height, m
dheight = 100 #deployment height, m
rho = 1.225 #air density, kg/m^3
deploy_time = 0.3 #parachute 2 deployment period
collide_time = 0.1 #rocket collision time, s

t_max = 6000 #maximum time
n_steps = int((t_max - t0) / dt) + 1 # Total number of steps

# Arrays to save simulation information

time = np.zeros(n_steps)
position = np.zeros(n_steps)
velocity = np.zeros(n_steps)
drag = np.zeros(n_steps)
acc = np.zeros(n_steps)
instantforce = np.zeros(n_steps)

# Initial properties: Rocket

y1_initial = height # m
v1_initial = 0  # m/s
acc_initial = 0 #ms^2
force_initial = 0 #N
drag_initial = 0 #N
time[0] = t0 #s

position[0] = y1_initial #m
velocity[0] = v1_initial  #m/s
acc[0] = acc_initial #m/s^2
instantforce[0] = force_initial #Force on rocket at apogee, N
drag[0] = drag_initial # Drag force on rocket at apogee, N

gforce = mass * 9.81 # Force of gravity

# Fixed Initial properties: Parachute 1

cod1 = 0.7
area1 = 5

# Variable initial properties: Parachute 2

cod2 = cod2
area2 = area2
counter = 0
termvel = -np.sqrt(2*mass*9.81/(rho*area2*cod2))

#======== PART 0.5: AFTER PARACHUTE 1 DEPLOYED ==================

counter = 0
```

```
    for i in range(1,n_steps):
        drag[i] = 0.5 * rho * cod1 * area1 * velocity[i-1]**2
        instantforce[i] = -gforce + drag[i]
        acc[i] = instantforce[i]/mass

        velocity[i] = velocity[i-1] + acc[i] * dt
        position[i] = position[i-1] + velocity[i] * dt

        time[i]  =  time[i-1] + dt
        counter = counter + 1
        if position[i] <= dheight: # If reaches deployment height
            break

    #======== PART I: PARACHUTE 2 DEPLOYMENT ==================

    mi = mass * velocity[counter-1]
    mf = mass * termvel
    dm = abs(mi - mf)
    force = dm/deploy_time

    print("====================SUMMARY OF RESULTS==================== \n")
    # Parachute 2 Deployment Failure

    if abs(force) > thresh_force:
        fig, axes = plt.subplots(1, 2, figsize=(10, 5))
        plt.subplots_adjust(left=0.08, right=0.95, top=0.82, bottom=0.12, wspace=0.3)

        axes[0].plot(time[:counter+1], velocity[:counter+1], color="brown", linewidth=2)
        axes[0].set_xlim(0, time[counter] + 5)
        axes[0].set_title("Rocket Velocity vs. Time", fontweight="bold", fontsize=12)
        axes[0].set_xlabel("Time (s)", fontsize=10)
        axes[0].set_ylabel("Velocity (m/s)", fontsize=10)
        axes[0].grid(True, alpha=0.3)
        axes[0].text(time[counter]/2, velocity[counter]/2, 'Recovery Fail:\nRocket Damaged',
                    horizontalalignment='center', verticalalignment='center', fontsize=11, color='black'
                    bbox=dict(boxstyle='round,pad=0.5', fc='red', ec='black', lw=2, alpha=0.5))

        axes[1].plot(time[:counter+1], position[:counter+1], color="orange", linewidth=2)
        axes[1].set_xlim(0, time[counter] + 5)
        axes[1].set_title("Rocket Position vs. Time", fontweight="bold", fontsize=12)
        axes[1].set_xlabel("Time (s)", fontsize=10)
        axes[1].set_ylabel("Position (m)", fontsize=10)
        axes[1].grid(True, alpha=0.3)
        axes[1].text(time[counter]/2, position[counter]/2, 'Recovery Fail:\nRocket Damaged',
                    horizontalalignment='center', verticalalignment='center', fontsize=11, color='black'
                    bbox=dict(boxstyle='round,pad=0.5', fc='red', ec='black', lw=2, alpha=0.5))

        fig.suptitle('Rocket Recovery Simulation', fontsize=16, fontweight='bold', y=0.98)
        fig.text(0.5, 0.90, 'Status: Failed during Parachute 2 Deployment',
                ha='center', fontsize=13, color='darkred', fontweight='bold')

        print("Collision 1 Failed: Rocket experienced force greater than threshold during Parachute 2 de
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {abs(force): 0.2f} N.")

        plt.tight_layout(rect=[0, 0, 1, 0.88])
        plt.show()
        counter2 = counter


    # Parachute 2 deployment success

    else:
        print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 depl
        dt = 1
        counter2 = counter

    #======== PART II: CONSTANT DESCENT TO GROUND ==================

        for i in range(counter, n_steps):
            position[i] = position[i-1] + termvel * dt
            counter2 = counter2 + 1
            time[i] = time[i-1] + dt
            velocity[i] = termvel
```

```
        if position[i] <= 0:
            velocity[i] = 0
            position[i] = 0
            break


#======== PART III: GROUND COLLISION ==================

    m11 = mass * velocity[counter2-2]
    force2 = m11/collide_time

# GROUND COLLISION FAILURE
    if abs(force2) >= thresh_force:
        print("Collision 2 Fail: Rocket experienced force greater than threshold during ground colli
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {abs(force): 0.2f} N.")
        print(f"The force during the ground collision was {abs(force2): 0.2f} N.")

        fig, axes2 = plt.subplots(1, 2, figsize=(10, 5))
        plt.subplots_adjust(left=0.08, right=0.95, top=0.78, bottom=0.1, wspace=0.3)

        axes2[0].plot(time[:counter2], velocity[:counter2], color="brown", linewidth=2)
        axes2[0].set_xlim(0, (time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight="bold", fontsize=12)
        axes2[0].set_xlabel("Time (s)", fontsize=10)
        axes2[0].set_ylabel("Velocity (m/s)", fontsize=10)
        axes2[0].grid(True, alpha=0.3)
        axes2[0].text(time[counter-1], velocity[counter-1]*0.5, 'Success:\nChute 2 Deployed',
                    horizontalalignment='center', verticalalignment='center', fontsize=10, color='bl
                    bbox=dict(boxstyle='round,pad=0.4', fc='green', ec='black', lw=2, alpha=0.5))
        axes2[0].text(time[counter2-1]*0.85, 0.5*(velocity[counter2-1]+velocity[counter2-2])*1.5, 'F
                    horizontalalignment='center', verticalalignment='bottom', fontsize=10, color='bl
                    bbox=dict(boxstyle='round,pad=0.4', fc='red', ec='black', lw=2, alpha=0.5))

        axes2[1].plot(time[:counter2], position[:counter2], color="orange", linewidth=2)
        axes2[1].set_xlim(0, (time[counter2-1] // 50)*50 + 50)
        axes2[1].set_title("Rocket Position vs. Time", fontweight="bold", fontsize=12)
        axes2[1].set_xlabel("Time (s)", fontsize=10)
        axes2[1].set_ylabel("Position (m)", fontsize=10)
        axes2[1].grid(True, alpha=0.3)
        axes2[1].text(time[counter-1]/2, (position[0] + position[counter-1])/2, 'Success:\nChute 2 D
                    horizontalalignment='center', verticalalignment='center', fontsize=10, color='bl
                    bbox=dict(boxstyle='round,pad=0.4', fc='green', ec='black', lw=2, alpha=0.5))
        axes2[1].text(time[counter2-1]*0.85, position[counter2-1] + 20, 'Fail:\nDamaged While\nLandi
                    horizontalalignment='center', verticalalignment='center', fontsize=10, color='bl
                    bbox=dict(boxstyle='round,pad=0.4', fc="red", ec='black', lw=2, alpha=0.5))

        fig.suptitle('Rocket Recovery Simulation', fontsize=16, fontweight='bold', y=0.98)
        fig.text(0.5, 0.89, 'Part I: Parachute 2 Deployment - Success', ha='center', fontsize=11, co
        fig.text(0.5, 0.85, 'Part II: Rocket Reaching Earth - Success', ha='center', fontsize=11, co
        fig.text(0.5, 0.81, 'Part III: Rocket Landing Safely - FAIL', ha='center', fontsize=11,
                color='darkred', fontweight='bold')

        plt.tight_layout(rect=[0, 0, 1, 0.8])
        plt.show()

# GROUND COLLISION SUCCESS, ROCKET RECOVERY SUCCESS

    else:
        print("Collision 2 Success: Rocket experienced force less than threshold during ground colli
        fig, axes2 = plt.subplots(1, 2, figsize=(10, 5))
        plt.subplots_adjust(left=0.08, right=0.95, top=0.78, bottom=0.1, wspace=0.3)

        axes2[0].plot(time[:counter2], velocity[:counter2], color="brown", linewidth=2)
        axes2[0].set_xlim(0, (time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight="bold", fontsize=12)
        axes2[0].set_xlabel("Time (s)", fontsize=10)
        axes2[0].set_ylabel("Velocity (m/s)", fontsize=10)
        axes2[0].grid(True, alpha=0.3)
        axes2[0].text(time[counter-1], velocity[counter-1]*0.5, 'Success:\nChute 2 Deployed',
                    horizontalalignment='center', verticalalignment='center', fontsize=10, color='bl
                    bbox=dict(boxstyle='round,pad=0.4', fc='green', ec='black', lw=2, alpha=0.5))
        axes2[0].text(time[counter2-1]*0.85, velocity[counter2-2]*0.5, 'Success:\nLanded Safely',
                    horizontalalignment='center', verticalalignment='center', fontsize=10, color='bl
```

```
                        bbox=dict(boxstyle='round,pad=0.4', fc='green', ec='black', lw=2, alpha=0.5))

            axes2[1].plot(time[:counter2], position[:counter2], color="orange", linewidth=2)
            axes2[1].set_xlim(0, (time[counter2-1] // 50)*50 + 50)
            axes2[1].set_title("Rocket Position vs. Time", fontweight="bold", fontsize=12)
            axes2[1].set_xlabel("Time (s)", fontsize=10)
            axes2[1].set_ylabel("Position (m)", fontsize=10)
            axes2[1].grid(True, alpha=0.3)
            axes2[1].text(time[counter-1]/2, (position[0] + position[counter-1])/2, 'Success:\nChute 2 D
                        horizontalalignment='center', verticalalignment='center', fontsize=10, color='bl
                        bbox=dict(boxstyle='round,pad=0.4', fc='green', ec='black', lw=2, alpha=0.5))
            axes2[1].text(time[counter2-1]*0.85, position[0]*0.15, 'Success:\nLanded Safely',
                        horizontalalignment='center', verticalalignment='center', fontsize=10, color='bl
                        bbox=dict(boxstyle='round,pad=0.4', fc="green", ec='black', lw=2, alpha=0.5))

            fig.suptitle('Rocket Recovery Simulation', fontsize=16, fontweight='bold', y=0.98)
            fig.text(0.5, 0.89, 'Part I: Parachute 2 Deployment - Success', ha='center', fontsize=11, co
            fig.text(0.5, 0.85, 'Part II: Rocket Reaching Earth - Success', ha='center', fontsize=11, co
            fig.text(0.5, 0.81, 'Part III: Rocket Landing Safely - SUCCESS!', ha='center', fontsize=11,
                    color='darkgreen', fontweight='bold')

            plt.tight_layout(rect=[0, 0, 1, 0.8])
            plt.show()

            print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2
            print("Collision 2 Success: Rocket experienced less force than threshold during ground colli
            print(f"The threshold force was {thresh_force: 0.2f} N.")
            print(f"The force during parachute 2 deployment was {abs(force): 0.2f} N.")
            print(f"The force during the ground collision was {abs(force2): 0.2f} N.")

    trialvel = velocity.copy()
    trialposition = position.copy()
    trialtime = time.copy()
    trialcount = counter2

    return trialvel, trialposition, trialtime, trialcount
```

Code Block Summary. The previous code in attempt 1 is packaged into a function.

In [34]:
```
# Run simulation with A = 25 m^2 and Cd = 1.25

trial2vel, trial2position, trial2time, trial2count = run_simulation(25, 1.25)
```

```
====================SUMMARY OF RESULTS====================

Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployment
Collision 2 Fail: Rocket experienced force greater than threshold during ground collision

The threshold force was  3500.00 N.
The force during parachute 2 deployment was  2778.52 N.
The force during the ground collision was  4192.79 N.
```

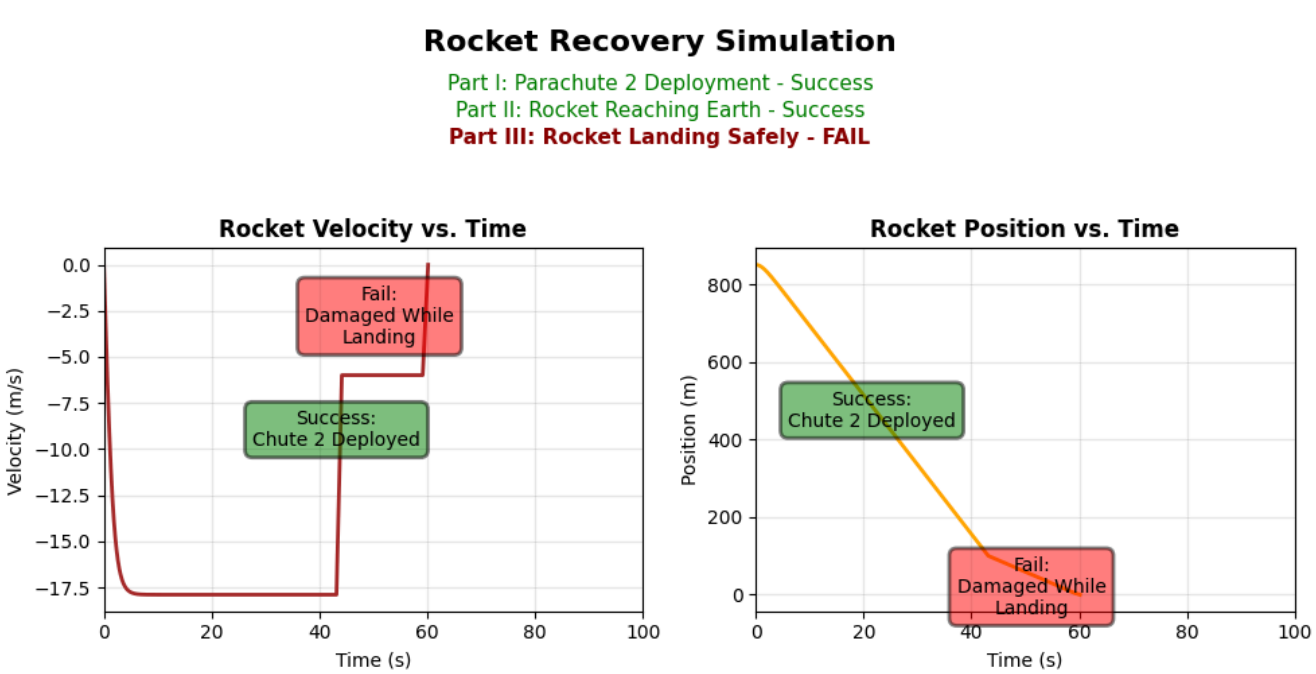


Figure 2. Recovery fail occurs when colliding with the ground, given Coefficient of Drag = 1.25, Surface Area = 25 m/s^2. The force during parachute 2 deployment was 2778.52 N, while the ground collision force was 4192.79 N. The threshold force is 3500 N. While parachute 2 is successfully deployed, the force by the ground onto the rocket exceeds the threshold force. This shows that an overly low coefficient of drag and/or surface area results in a parachute 2 deployment success but a ground collision fail, due to a lack of deceleration.

Note that $t = 0$ corresponds to the apogee (peak height), and that ascent data are not included.

Attempt 2: Analysis

In the velocity vs. time graph, the trajectory of the rocket from t=0 to t=40s is very similar: in the Rocket vs. Velocity time graph, we first see the rocket reaching Parachute 1's terminal velocity. Notice how the time taken to reach terminal velocity is shorter than in Attempt 2; this is expected, since the coefficient of drag and surface area have been decreased, lowering its corresponding terminal velocity. This allows the rocket to reach the velocity more quickly.

After t=40, we notice that there is a sudden drop in velocity in the Rocket Velocity vs. Time graph. This is indicative of parachute 2 being deployed, and the rocket transitioning into parachute 2's much lower terminal velocity. Since it is so low, the rocket immediately reaches parachute 2's terminal velocity and stays constant until collision with the ground. In the Rocket vs. Position Time graph, this can be seen by the abrupt change in slope at t=600.

However, even with these changes, the rocket recovery is a failure since it is damaged while landing; the force during ground collision was 4193N, which far exceeds the threshold force of 693N.

Hence, to improve upon this attempt, we should slightly increase the coefficient of drag and/or area of Parachute 2, as to soften the rocket's ground collision, but to still retain Parachute 2's successful deployment.

Attempt 3: Coefficient of Drag = 1.8, Surface Area = 25 m^2

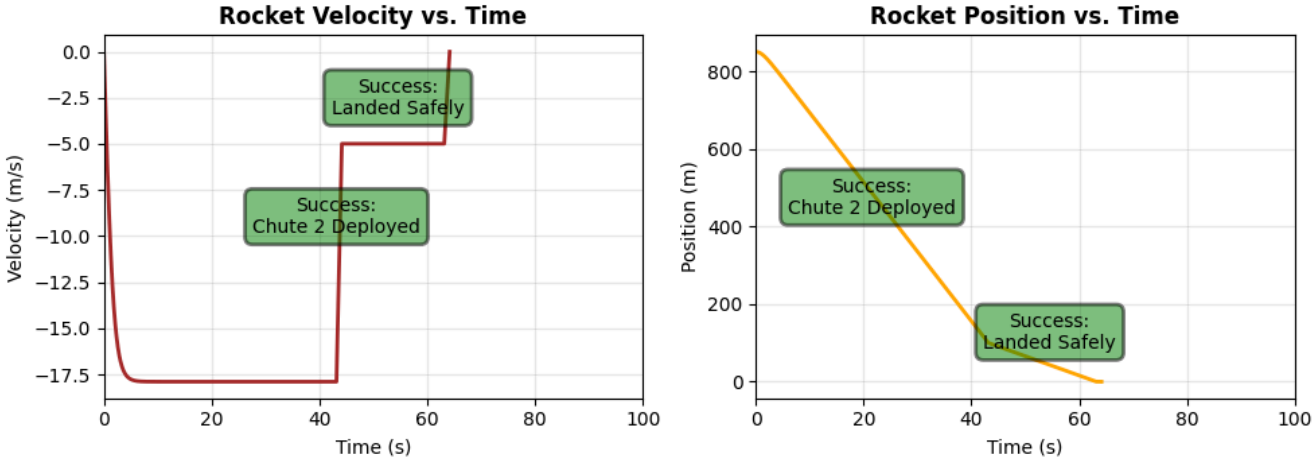
From Attempt 2, we slightly increase the coefficient of drag in Parachute 2. We analyze and plot Velocity vs. Time and Position vs. Time graphs when the $C_D = 1.8$ and $A = 25m^2$.

In [35]:
```
# Run simulation with A = 25 m^2 and Cd = 1.8

trial3vel, trial3position, trial3time, trial3count = run_simulation(25, 1.8)
```

```
====================SUMMARY OF RESULTS====================

Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployment
Collision 2 Success: Rocket experienced force less than threshold during ground collision
```

Rocket Recovery Simulation

Part I: Parachute 2 Deployment - Success
Part II: Rocket Reaching Earth - Success
Part III: Rocket Landing Safely - SUCCESS!




```
Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployment
Collision 2 Success: Rocket experienced less force than threshold during ground collision

The threshold force was  3500.00 N.
The force during parachute 2 deployment was  3011.46 N.
The force during the ground collision was  3493.99 N.
```

Figure 3. Recovery success occurs when coefficient of drag is 1.8 and cross-sectional surface area is 25 m^2; threshold force 3500 N, while parachute 2 deployment force was 3011.46 N, and collision force was 3493.99 N. Both Parachute 2 deployment and ground collision forces are less than 3500 N. This shows that a balance between a high and low coefficient of drag and/or surface area is needed to achieve a successfuly rocket recovery.

Note that $t = 0$ corresponds to the apogee (peak height), and that ascent data are not included.

Attempt 3 Analysis

Comparing Attempt 2 to Attempt 3, the Rocket Velocity vs. Time and Rocket Position vs. Time graphs are very similar. In the Rocket Velocity vs Time grph, the rocket quickly reaches Parachute 1's terminal velocity, and at t=40, suddenly drops to that of Parachute 2, staying at constant velocity until it collides with the ground. The forces during both Parachute 2 and Ground Collision is both less than 3500 N, creating a successful rocket landing.

Preliminary Trial General Analysis, Without Wind

In [36]:
```
# Vertically Stacking Three Trials

fig, axes3 = plt.subplots(1, 2)
plt.subplots_adjust(wspace=0.5)
fig.set_size_inches(14, 5)
fig.suptitle('Rocket Recovery Graphs (Vertically Stacking Three Trials)', fontsize=16, fontweight='bold'
plt.subplots_adjust(top=0.85)

axes3[0].plot(trial1time[:trial1count], trial1vel[:trial1count], c = "steelblue", label = "Trial 1 - Chu
axes3[0].plot(trial2time[:trial2count], trial2vel[:trial2count], c = "mediumpurple", label = "Trial 2 -
axes3[0].plot(trial3time[:trial3count], trial3vel[:trial3count], c = "palevioletred", label = "Trial 3 -
axes3[0].set_xlim(0, 70)
axes3[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
axes3[0].set_xlabel("Time (s)")
axes3[0].set_ylabel("Velocity (m/s)")
axes3[0].legend()

axes3[1].plot(trial1time[:trial1count], trial1position[:trial1count], c = "steelblue", label = "Trial 1
axes3[1].plot(trial2time[:trial2count], trial2position[:trial2count], c = "mediumpurple", label = "Trial
axes3[1].plot(trial3time[:trial3count], trial3position[:trial3count], c = "palevioletred", label = "Tria
axes3[1].set_xlim(0, 70)
axes3[1].set_title("Rocket Position vs. Time", fontweight = "bold")
axes3[1].set_xlabel("Time (s)")
```

```
axes3[1].set_ylabel("Position (m)")
axes3[1].legend()
```

Out[36]: <matplotlib.legend.Legend at 0x12669ad00>

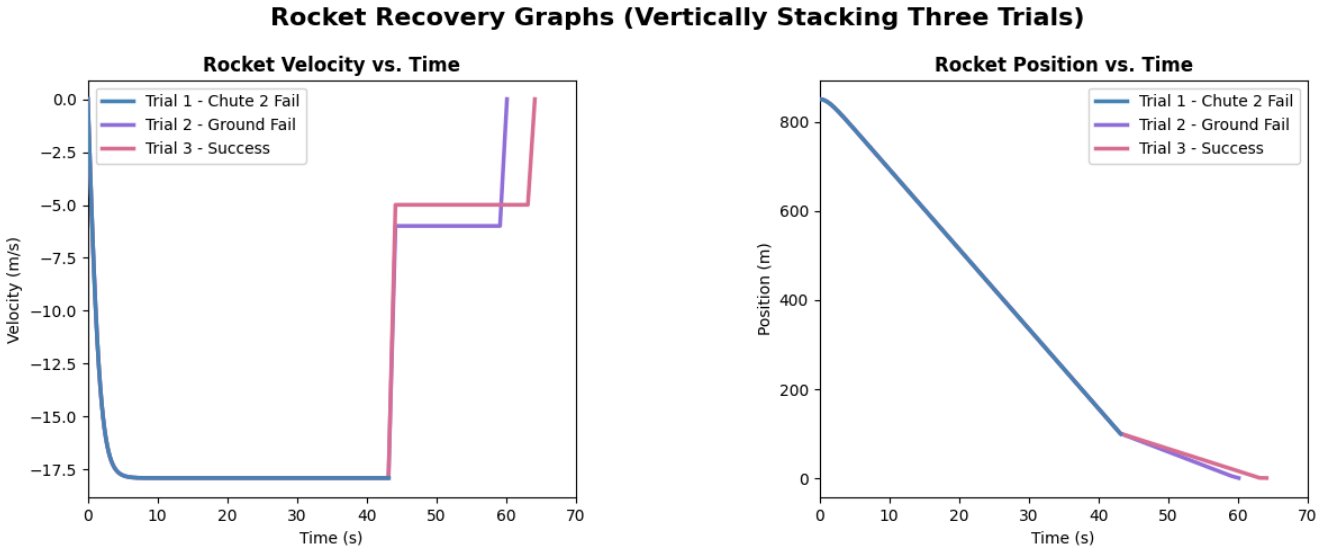


Figure 4. Vertically stacked velocity vs time and position vs. time graphs of all three trials.

Summary of Preliminary Trials With Wind (Threshold Force = 3500 N)

- **Attempt 1 ($C_d = 1.5, A = 130m^2$)**: Parachute deployment force = 3616.64 N
- **Attempt 2 ($C_d = 1.25, A = 25m^2$)**: Parchute deployment force = 2778.52 N, ground collision force = 4192.79 N
- **Attempt 3 ($C_d = 1.8, A = 25m^2$)**: Parachute deployment force = 3011.46 N, ground collision force = 3493.99 N.

From these preliminary trials, the following general observations can be made:

- **Large**: If the coefficient of drag and surface area of Parachute 2 is large, the rocket will be damaged during Parachute 2 deployment. This is because the terminal velocity of Parachute 2 will be extremely low, and the momentum arising from Parachute 1 to Parachute 2's change in terminal velocities will create a force over 3500N on the rocket
- **Small**: Conversely, if the coefficient of drag and surface area of Parachute is small, the rocket will be damaged during its ground collision. A low C_D and surface area corresponds with a higher terminal velocity; so, when the Parachute loses all its momentum in 0.01s, this similarly creates a massive force over 3500N on the rocket.
- **The "Sweet Spot"**: To find the conditions that both deploys Parachute 2 and lands the rocket safely, a "sweet spot" needs to be found in the middle between large and small drag coefficients and surface areas.

While the Preliminary Trials gives a vague ballpark of what these values may be, more technical analysis should be conducted to exactly identify the range of parameters that yields a successful recovery. Part 3 of this project's analysis does this.

2.2 Preliminary Trials With Wind

Next, we extend upon portion (1) of the analysis by implementing a variable initial launch angle and constant wind. As detailed in "Programming", the wind introduce a difference between the rocket's relative velocity with the air compared to that of the ground. When calculating drag, we take the former. We assume that the rocket does not experience drag in its ascent. We take wind to be $3\text{m}/\text{s}$.

We first very initial launch angle to understand its behaviour on the rocket. We then follow the same procedure as portion (1) of the analysis, where we find combinations of launch angle, drag coefficients, and surface area that lead to the rocket landing safely. This allows as to validate the preliminary conclusions made in portion (1). Note that in the latter portion, we do not consider the rocket's final position yet; this will be explored in part (4).

Cycle 1: Varying Launch Angle

We first set launch angle to be 45, 85, 90, and 130 degrees and observe rocket trajectory behaviour. We fix the coefficient of drag to be 2 and surface rea to be 25 m^2.

In [37]:

```
import numpy as np
import matplotlib.pyplot as plt
import math
from mpl_toolkits.mplot3d import Axes3D

## USER INPUTS_THRESHOLD FORCE AND THRUST FORCE
thrust_force = 3000
thresh_force = 3500
wind = 3 # m/s wind speed acting against rocket during descent

##### LAUNCHING

def get_trajectory(cod2, area2, angle):

    ## CREATING EMPTY LISTS AND INITIAL VALUES FOR EACH VARIABLE
    rho = 1.225 #air density, kg/m^3
    mass = 70 # Rocket kg
    forcex_launch = []
    forcey_launch = []

    x_launch = []
    x_launch.append(0)

    y_launch = []
    y_launch.append(0)

    accx_launch = []
    accy_launch = []

    vx_launch = []
    vx_launch.append(0)

    vy_launch = []
    vy_launch.append(0)

    t_launch = []
    t_launch.append(0)
    dt_launch = 0.01
    gforce = mass * 9.81 #N, gravitational force

    angle = angle

    # Fixed Initial properties: Parachute 1
    cod1 = 0.7 #coefficient of drag
    area1 = 5 #cross-sectional surface area, m^2

    # Variable Properties: Parachute 2
    cod2 = cod2  #coefficient of drag
    area2 = area2  #cross-sectional surface area, m^2
    termvel = np.sqrt(2*mass*9.81/(rho*area2*cod2))

    rocketarea = 0 #for drag during ascent
    streamlinecod = 0

    print("====================SUMMARY OF RESULTS==================== \n")
    ## PART 0: ASCENT TO APOGEE

    j = 0
    while True:
        # Thrust forces
        if t_launch[j] <= 5:
            xthrust = thrust_force * math.cos(math.radians(angle))
            ythrust = thrust_force * math.sin(math.radians(angle))
        else:
            xthrust = 0
            ythrust = 0

        # Drag forces during ascent
        if j > 0:
            v_rel_x = vx_launch[j] - wind
            v_rel_y = vy_launch[j] - 0  # no vertical wind
            v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)
```

```
            if v_rel_mag > 0.01:  # avoid division by zero

                # No drag for rocket ascending

                drag_magnitude = 0.5 * rho * rocketarea * streamlinecod * v_rel_mag**2
                xdrag = -drag_magnitude * v_rel_x / v_rel_mag
                ydrag = -drag_magnitude * v_rel_y / v_rel_mag
            else:
                xdrag = 0
                ydrag = 0
        else:
            xdrag = 0
            ydrag = 0

        # Net forces = thrust + drag - weight
        forcex_launch.append(xthrust + xdrag)
        forcey_launch.append(ythrust + ydrag - gforce)

        accx_launch.append(forcex_launch[j]/mass)
        accy_launch.append(forcey_launch[j]/mass)

        vx_launch.append(vx_launch[j] + accx_launch[j]*dt_launch)
        vy_launch.append(vy_launch[j] + accy_launch[j]*dt_launch)

        x_launch.append(x_launch[j] + vx_launch[j]*dt_launch)
        y_launch.append(y_launch[j] + vy_launch[j]*dt_launch)

        t_launch.append(t_launch[j] + dt_launch)
        j = j + 1

        if vy_launch[j] <= 0: # If reaches apogee
            break

    ######## RECOVERY

    ## SET INITIAL VARIABLES

    t0 = 0      # starting time, s
    dt = 0.01   # time step, s
    height = y_launch[j] #apogee height, m

    apogeeindex = j

    dheight = 500 #deployment height, m
    rho = 1.225 #air density, kg/m^3
    deploy_time = 0.3 #parachute 2 deployment period
    collide_time = 0.1 #rocket collision time, s

    t_max = 6000 #maximum time
    n_steps = int((t_max - t0) / dt) + 1

    ## Arrays to save simulation information
    time = np.zeros(n_steps) #s
    xposition = np.zeros(n_steps) #m
    yposition = np.zeros(n_steps) #m

    xvelocity = np.zeros(n_steps) #m/s
    yvelocity = np.zeros(n_steps) #m/s

    xdrag = np.zeros(n_steps) #N
    ydrag = np.zeros(n_steps) #N

    xacc = np.zeros(n_steps) #m/s^2
    yacc = np.zeros(n_steps) #m/s^2

    xinstantforce = np.zeros(n_steps) #N
    yinstantforce = np.zeros(n_steps) #N
    yinstantforce[0] = -gforce

    yposition[0] = height #m
    yvelocity[0] = vy_launch[j] #m/s

    xposition[0] = x_launch[j] #m
    xvelocity[0] = vx_launch[j] #m/s
```

```
time[0] = 0  #s

xdrag[0] = 0 #N
ydrag[0] = 0 #N

xacc[0] = 0 #m/s^2
yacc[0] = -9.81 #m/s^2

# PART 0.5: AFTER PARACHUTE 1 DEPLOYED

for i in range(1, n_steps):
    # Calculate relative velocity
    v_rel_x = xvelocity[i-1] - wind
    v_rel_y = yvelocity[i-1] - 0  # no vertical wind
    v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)

    # Calculate drag force components
    if v_rel_mag > 0.01:
        drag_magnitude = 0.5 * rho * cod1 * area1 * v_rel_mag**2
        xdrag[i] = -drag_magnitude * v_rel_x / v_rel_mag
        ydrag[i] = -drag_magnitude * v_rel_y / v_rel_mag
    else:
        xdrag[i] = 0
        ydrag[i] = 0

    # Net forces
    xinstantforce[i] = xdrag[i]
    yinstantforce[i] = ydrag[i] - gforce  # weight pulls down

    # Acceleration
    xacc[i] = xinstantforce[i] / mass
    yacc[i] = yinstantforce[i] / mass

    # Update velocities
    xvelocity[i] = xvelocity[i-1] + xacc[i] * dt
    yvelocity[i] = yvelocity[i-1] + yacc[i] * dt

    # Update positions
    xposition[i] = xposition[i-1] + xvelocity[i] * dt
    yposition[i] = yposition[i-1] + yvelocity[i] * dt

    time[i] = time[i-1] + dt

    if yposition[i] <= dheight:
        break

# Parachute 2 Deployment Phase
momentumix = mass * xvelocity[i]
momentumiy = mass * yvelocity[i]

# Terminal velocity targets
xvel = wind  # horizontal velocity matches wind
yvel = -termvel  # negative because falling down
momentumfx = mass * xvel
momentumfy = mass * yvel

fx = (momentumfx - momentumix) / deploy_time
fy = (momentumfy - momentumiy) / deploy_time
fdeploy = math.sqrt(fx**2 + fy**2)

# If deployment force exceeds threshold, parachute fails to deploy
if fdeploy > thresh_force:
    time_offset = t_launch[-1]
    full_time = t_launch + [time_offset + time[k] for k in range(i+1)]
    full_x = x_launch + list(xposition[:i+1])
    full_y = y_launch + list(yposition[:i+1])
    print(f"Parachute Deployment Failed. Deployment Force: {abs(fdeploy):.0f} N")
    print(f"Threshold Force: {thresh_force} N")
    fcollide = 0
    return full_time, full_x, full_y, "Deployment Failed", apogeeindex, fdeploy, fcollide

else:
    # Descent to ground at constant velocity
```

```
        for j in range(i+1, n_steps):
            xvelocity[j] = xvel
            yvelocity[j] = yvel

            xposition[j] = xposition[j-1] + xvelocity[j] * dt
            yposition[j] = yposition[j-1] + yvelocity[j] * dt
            time[j] = time[j-1] + dt

            if yposition[j] <= 0:
                xposition[j] = xposition[j-1]
                yposition[j] = 0
                break

        # PART III: Ground Collision Phase
        momentum = mass * yvelocity[j]
        fcollide = abs(momentum / collide_time)

        # Crash Landing
        if fcollide > thresh_force:
            time_offset = t_launch[-1]
            full_time = t_launch + [time_offset + time[k] for k in range(j+1)]
            full_x = x_launch + list(xposition[:j+1])
            full_y = y_launch + list(yposition[:j+1])
            print(f"Parachute Deployment Success. Deployment Force: {abs(fdeploy):.0f} N")
            print(f"Crash Landing. Collision Force: {fcollide:.0f} N")
            print(f"Threshold Force: {thresh_force} N")
            return full_time, full_x, full_y, "Crash Landing", apogeeindex,  fdeploy, fcollide

        # Safe landing
        else:
            time_offset = t_launch[-1]
            full_time = t_launch + [time_offset + time[k] for k in range(j+1)]
            full_x = x_launch + list(xposition[:j+1])
            full_y = y_launch + list(yposition[:j+1])
            print(f"Parachute Deployment Success. Deployment Force: {abs(fdeploy):.0f} N")
            print(f"Safe Landing. Collision Force: {fcollide:.0f} N")
            print(f"Threshold Force: {thresh_force} N")
            return full_time, full_x, full_y, "Safe Landing", apogeeindex,  fdeploy, fcollide

import numpy as np
import matplotlib.pyplot as plt
import math
from mpl_toolkits.mplot3d import Axes3D

def plot_all_trajectories(time_data, x_data, y_data, apogee_index, status, fdeploy, fcollide):

    # Determine status box color
    if status == "Deployment Failed":
        boxcolor = "red"
    elif status == "Crash Landing":
        boxcolor = "chocolate"
    elif status == "Safe Landing":
        boxcolor = "green"
    else:
        boxcolor = "gray"

    # Create figure with 2x2 subplots
    fig = plt.figure(figsize=(20, 8))

    # Common plot settings
    trajectory_color = "#9b2864"
    x_proj_color = "#4a0b6b"
    y_proj_color = "#fcb017"


    # ========== SUBPLOT 1: FULL TRAJECTORY ==========

    ax1 = fig.add_subplot(1, 3, 1, projection='3d')

    ax1.plot(time_data, x_data, y_data, '-', linewidth=2, color=trajectory_color, label='Rocket Trajecto
    ax1.scatter(time_data[0], x_data[0], y_data[0], color='navy', s=100, label='Launch', marker='o')
    ax1.scatter(time_data[apogee_index], x_data[apogee_index], y_data[apogee_index],
                c=y_proj_color, s=100, label='Apogee', marker='o')
```

```
    # Projections
    ax1.plot(time_data, x_data, [min(y_data)]*len(time_data), '--', c=x_proj_color,
             linewidth=1.25, alpha=1, label='X Projection')
    ax1.plot(time_data, [min(x_data)]*len(time_data), y_data, '--', c=y_proj_color,
             linewidth=1.25, alpha=1, label='Y Projection')

    ax1.set_xlabel('Time (s)', fontsize=10, labelpad=10)
    ax1.set_ylabel('X Position (m)', fontsize=10, labelpad=10)
    ax1.set_zlabel('Y Position (m)', fontsize=10, labelpad=10)
    ax1.set_title('Full Trajectory', fontsize=12, fontweight='bold', pad=10)
    ax1.legend(fontsize=8, loc='upper right')
    ax1.view_init(elev=20, azim=45)
    ax1.grid(True, alpha=0.3)

    # Status box
    ax1.text2D(0.05, 0.95, f'Status: {status}', transform=ax1.transAxes,
               fontsize=9, verticalalignment='top',
               bbox=dict(boxstyle='round,pad=0.5', fc=boxcolor, ec='black', lw=2, alpha=0.5))

    # ========== SUBPLOT 2: LAUNCH PHASE ==========
    ax2 = fig.add_subplot(1, 3, 2, projection='3d')

    ax2.plot(time_data[0:apogee_index], x_data[0:apogee_index], y_data[0:apogee_index],
             '-', linewidth=2, color=trajectory_color, label='Rocket Trajectory')
    ax2.scatter(time_data[0], x_data[0], y_data[0], color='navy', s=100, label='Launch', marker='o')
    ax2.scatter(time_data[apogee_index], x_data[apogee_index], y_data[apogee_index],
                c=trajectory_color, s=100, label=f'Apogee Point', marker='o')

    # Projections
    ax2.plot(time_data[0:apogee_index], x_data[0:apogee_index],
             [min(y_data)]*len(time_data[0:apogee_index]), '--', c=x_proj_color,
             linewidth=1.25, alpha=1, label='X Projection')
    ax2.plot(time_data[0:apogee_index], [min(x_data)]*len(time_data[0:apogee_index]),
             y_data[0:apogee_index], '--', c=y_proj_color, linewidth=1.25, alpha=1, label='Y Projection'

    ax2.set_xlabel('Time (s)', fontsize=10, labelpad=10)
    ax2.set_ylabel('X Position (m)', fontsize=10, labelpad=10)
    ax2.set_zlabel('Y Position (m)', fontsize=10, labelpad=10)
    ax2.set_title('Launch Phase', fontsize=12, fontweight='bold', pad=10)
    ax2.legend(fontsize=8, loc='upper left')
    ax2.view_init(elev=20, azim=45)
    ax2.grid(True, alpha=0.3)

    # ========== SUBPLOT 3: APOGEE REGION ==========
    ax3 = fig.add_subplot(1, 3, 3, projection='3d')

    min_scale = 0.5
    max_scale = 1.5
    start_idx = int(apogee_index * min_scale)
    end_idx = int(apogee_index * max_scale)

    ax3.plot(time_data[start_idx:end_idx], x_data[start_idx:end_idx], y_data[start_idx:end_idx],
             '-', linewidth=2, color=trajectory_color, label='Rocket Trajectory')
    ax3.scatter(time_data[apogee_index], x_data[apogee_index], y_data[apogee_index],
                c=y_proj_color, s=100, label='Apogee', marker='o')
    ax3.scatter(time_data[start_idx], x_data[start_idx], y_data[start_idx],
                c=trajectory_color, s=80, label=f'Start Window', marker='o')
    ax3.scatter(time_data[end_idx], x_data[end_idx], y_data[end_idx],
                c=trajectory_color, s=80, label=f'End Window', marker='o')

    # Projections
    ax3.plot(time_data[start_idx:end_idx], x_data[start_idx:end_idx],
             [min(y_data)]*len(time_data[start_idx:end_idx]), '--', c=x_proj_color,
             linewidth=1.25, alpha=1, label='X Projection')
    ax3.plot(time_data[start_idx:end_idx], [min(x_data)]*len(time_data[start_idx:end_idx]),
             y_data[start_idx:end_idx], '--', c=y_proj_color, linewidth=1.25, alpha=1, label='Y Projecti

    ax3.set_xlabel('Time (s)', fontsize=10, labelpad=10)
    ax3.set_ylabel('X Position (m)', fontsize=10, labelpad=10)
    ax3.set_zlabel('Y Position (m)', fontsize=10, labelpad=10)
    ax3.set_title('Apogee Region', fontsize=12, fontweight='bold', pad=10)
    ax3.legend(fontsize=8, loc='upper left')
    ax3.view_init(elev=20, azim=45)
    ax3.grid(True, alpha=0.3)
```

```
    # ========== OVERALL TITLE AND DETAILS ==========
    fig.suptitle('3D Rocket Trajectory Analysis: Multiple Views', fontsize=16, fontweight='bold', y=0.85

    # Add details text
    if status == "Deployment Failed":
        details_text = f'Apogee: {max(y_data):.0f} m  |  Deployment Force: {fdeploy:.0f} N  |  Threshold
    else:
        details_text = f'Apogee: {max(y_data):.0f} m  |  Deployment Force: {fdeploy:.0f} N  |  Collision

    fig.text(0.5, 0.16, details_text, ha='center', fontsize=11,
             bbox=dict(boxstyle='round,pad=0.5', fc='silver', ec='black', lw=2, alpha=0.5))

    plt.tight_layout(rect=[0, 0.03, 1, 0.97])
    plt.show()
```

Trial 1: $\theta = 45^o$

In [38]:
```
# ANGLE = 40 Degrees

cod2 = 2  # Coefficient of drag for parachute 2
area2 = 25  # Cross-sectional area for parachute 2 in m^2
angle = 45

# MAIN FUNCTION EXECUTION
time_data, x_data, y_data, status, apogeeindex, fdeploy, fcollide = get_trajectory(cod2, area2, angle) #

#plot_all_trajectories(time_data, x_data, y_data, apogeeindex, status, fdeploy, fcollide)
```

```
====================SUMMARY OF RESULTS====================

Parachute Deployment Success. Deployment Force: 3071 N
Safe Landing. Collision Force: 3315 N
Threshold Force: 3500 N
```

Code Block Summary: The above code extends on the code from "Preliminary Trials without Wind" and takes account of the wind moving at $3m/s$ in the positive x-direction. In the updated code, the total drag is calculated based on the rocket's velocity relative to the air (wind). The x and y components of drag is then calculated given the angle made by the relative x and y velocity vectors.

In [ ]:
```

```


45

Figure 5. Rocket trajectory Velocity vs. Time and Position vs. Time graphs when $\theta = 45^o$

Trial 2: $\theta = 85^o$

In [39]:
```
# ANGLE = 80 Degrees

cod2 = 2  # Coefficient of drag for parachute 2
area2 = 25  # Cross-sectional area for parachute 2 in m^2
angle = 85

# MAIN FUNCTION EXECUTION
time_data, x_data, y_data, status, apogeeindex, fdeploy, fcollide = get_trajectory(cod2, area2, angle) #

#plot_all_trajectories(time_data, x_data, y_data, apogeeindex, status, fdeploy, fcollide)
```

```
====================SUMMARY OF RESULTS====================

Parachute Deployment Success. Deployment Force: 3071 N
Safe Landing. Collision Force: 3315 N
Threshold Force: 3500 N
```


85

Figure 6. Rocket trajectory Velocity vs. Time and Position vs. Time graphs when $\theta = 85^o$

Trial 3: $\theta = 90^o$

In [40]:
```
#### Trial 3: $\theta = 90^o$

# ANGLE = 90 Degrees

cod2 = 2  # Coefficient of drag for parachute 2
area2 = 25  # Cross-sectional area for parachute 2 in m^2
angle = 90

# MAIN FUNCTION EXECUTION
time_data, x_data, y_data, status, apogeeindex, fdeploy, fcollide = get_trajectory(cod2, area2, angle) #

#plot_all_trajectories(time_data, x_data, y_data, apogeeindex, status, fdeploy, fcollide)
```

```
====================SUMMARY OF RESULTS====================

Parachute Deployment Success. Deployment Force: 3071 N
Safe Landing. Collision Force: 3315 N
Threshold Force: 3500 N
```

Figure 7. Rocket trajectory Velocity vs. Time and Position vs. Time graphs when $\theta = 90^o$.

Trial 4: $\theta = 130^o$

In [41]:
```
#### Trial 4: $\theta = 130^o$

cod2 = 2  # Coefficient of drag for parachute 2
area2 = 25  # Cross-sectional area for parachute 2 in m^2
angle = 130

# MAIN FUNCTION EXECUTION
time_data, x_data, y_data, status, apogeeindex, fdeploy, fcollide = get_trajectory(cod2, area2, angle) #

#plot_all_trajectories(time_data, x_data, y_data, apogeeindex, status, fdeploy, fcollide)
```

```
====================SUMMARY OF RESULTS====================

Parachute Deployment Success. Deployment Force: 3071 N
Safe Landing. Collision Force: 3315 N
Threshold Force: 3500 N
```

Figure 8. Rocket trajectory Velocity vs. Time and Position vs. Time graphs when $\theta = 130^o$.

Cycle 1 Analysis

- **Forces**: From this round of trials, it seems that launch angle does not change the deployment and collision force. This makes sense, assumiing that the rocket reaches parachute 1's terminal velocity and immediate drops to parachute 2's terminal velocity upon deployment.
- **Apogee**: Apogee drastically changes as initial angle is varied. Again, this is expected; as the angle approaches $\theta = 90^o$ (vertical), the apogee increases, since the rocket has more vertical velocity.
- **Landing Spot**: At acute angles, the rocket is displaced in the positive X direction, since there is a positive horizontal vertical component. At $\theta = 90^o$ (vertical), the rocket displacement is still positive; this is due to the wind direction betwen rightwards. Finally, at large obtuse angles, the rocket is displaced in the negative X direction. However, given thst position is positive at a vertical launch, we expect that at a specific small obtuse angle, the rocket can land at its original position, X=0.

Cycle 2: Varying Coefficient of Drag and Surface Area

Next, we follow the same methodology as portion (1) of analysis, where we test low values of coefficient of drag and surface areas to high values. We do this over three trials. Since we found that launch angle does not affect collision forces, we set $\theta = 80$ as a fixed value.

Trial 1: $C_D = 1.25, A = 130m^2$

In [42]:
```
#### Trial 1: $C_D = 1.25, A = 130 m^2$


cod2 = 1.25  # Coefficient of drag for parachute 2
area2 = 130 # Cross-sectional area for parachute 2 in m^2
angle = 80

# MAIN FUNCTION EXECUTION
time_data, x_data, y_data, status, apogeeindex, fdeploy, fcollide = get_trajectory(cod2, area2, angle) #

#plot_all_trajectories(time_data, x_data, y_data, apogeeindex, status, fdeploy, fcollide)
```

```
====================SUMMARY OF RESULTS====================

Parachute Deployment Failed. Deployment Force: 3563 N
Threshold Force: 3500 N
```

Figure 9. Rocket trajectory Velocity vs. Time and Position vs. Time graphs when $C_D = 1.25, A = 130m^2$.

Trial 2: $C_D = 1, A = 25m^2$

In [43]:
```
#### Trial 2: $C_D = 1, A = 25 m^2$

cod2 = 1  # Coefficient of drag for parachute 2
area2 = 25  # Cross-sectional area for parachute 2 in m^2
angle = 80

# MAIN FUNCTION EXECUTION
time_data, x_data, y_data, status, apogeeindex, fdeploy, fcollide = get_trajectory(cod2, area2, angle) #

#plot_all_trajectories(time_data, x_data, y_data, apogeeindex, status, fdeploy, fcollide)
```

```
====================SUMMARY OF RESULTS====================

Parachute Deployment Success. Deployment Force: 2614 N
Crash Landing. Collision Force: 4688 N
Threshold Force: 3500 N
```

Figure 10. Rocket trajectory Velocity vs. Time and Position vs. Time graphs when $C_D = 1, A = 25m^2$.

Trial 3: $C_D = 1.25, A = 40m^2$

In [44]:
```
#### Trial 3: $C_D = 1.24, A = 40 m^2$

cod2 = 1.25  # Coefficient of drag for parachute 2
area2 = 40  # Cross-sectional area for parachute 2 in m^2
angle = 80

# MAIN FUNCTION EXECUTION
time_data, x_data, y_data, status, apogeeindex, fdeploy, fcollide = get_trajectory(cod2, area2, angle) #

#plot_all_trajectories(time_data, x_data, y_data, apogeeindex, status, fdeploy, fcollide)
```

```
====================SUMMARY OF RESULTS====================

Parachute Deployment Success. Deployment Force: 3071 N
Safe Landing. Collision Force: 3315 N
Threshold Force: 3500 N
```

Figure 11. Rocket trajectory Velocity vs. Time and Position vs. Time graphs when $C_D = 1, A = 40m^2$.

Preliminary Trial With Wind Analysis

- **Cycle 1**: From cycle 1, we learn that angle does not affect the deployment and collision force of the rocket. This is assuming that the angle is within the range such that the terminal velocity of parachute 1 is still reached, and that the

apogee is above the deployment height of parachute 2.

- **Cycle 2**: From cycle 2, we validate our conclusions from portion (1) of analysis. When coefficient of drag and area is very large, the force on the rocket during parachute 2 deployment is too large, causing a deployment failure. On the contrary, when the coefficeint of drag and area is too small, the force on the rocket during ground collision is too large. The key to creating a safe landing is to find the middle region where both issues are avoided, as seen in Trial 3.

3. Quantitative Phase Space Scan: Determining Initial Angle, Drag Coefficient, and Areas for Successful Landing

In this section, we quantitatively determine the what drag coefficients and surfaces areas possible to create a safe rocket landing. We do this in two steps. In the first step, we only look at the combinations of drag coefficient and surface area that yield a safe landing. In the second step, we take the range of drag coefficients and surface areas that create a safe landing and find the subset of that range where the rocket also lands within 100m of its original position.

3.1 Step 1: Phase Space of Drag Coefficients and Surface Area to Create Safe Landings

We know that the drag force is

$$F_D = \frac{1}{2} C_D \rho A v^2$$

where C_D is the drag coefficient and A is cross-sectional surface area. We notice that drag force is direct proportional to C_D and A, with the same magnitude of impact, 1. So, create a new constant C, where

$$C = C_D \times A$$

Using this, we can calculate Parachute 2 deployment and ground collision force over a range of C, identifying the range of drag coefficients and surface area where both corresponding forces are under 3500N. We fix $\theta = 90^o$.

In [45]:
```
import numpy as np
import matplotlib.pyplot as plt

## USER INPUTS_THRESHOLD FORCE AND THRUST FORCE
thresh_force = 3500
thrust_force = 3000
wind = 3 # m/s wind speed acting against rocket during descent

chute_force = []
collision_force = []

##### LAUNCHING
arr = np.linspace(15, 200, 200)
chute_force = np.zeros(200)
collision_force = np.zeros(200)
b = 0
fifteen = []
i = 0

for constant in arr:
    ## CREATING EMPTY LISTS AND INITIAL VALUES FOR EACH VARIABLE
    rho = 1.225 #air density, kg/m^3
    mass = 70 # Rocket kg
    forcex_launch = []
    forcey_launch = []

    x_launch = []
    x_launch.append(0)

    y_launch = []
    y_launch.append(0)

    accx_launch = []
    accy_launch = []

    vx_launch = []
    vx_launch.append(0)
```

```
vy_launch = []
vy_launch.append(0)

t_launch = []
t_launch.append(0)
dt_launch = 0.01
gforce = mass * 9.81 #N, gravitational force

angle = angle

# Fixed Initial properties: Parachute 1
cod1 = 0.7 #coefficient of drag
area1 = 5 #cross-sectional surface area, m^2

# Variable Properties: Parachute 2
termvel = np.sqrt(2*mass*9.81/(rho*constant))

rocketarea = 0 #for drag during ascent
streamlinecod = 0

## PART 0: ASCENT TO APOGEE

j = 0
while True:
    # Thrust forces
    if t_launch[j] <= 5:
        xthrust = thrust_force * math.cos(math.radians(angle))
        ythrust = thrust_force * math.sin(math.radians(angle))
    else:
        xthrust = 0
        ythrust = 0

    # Drag forces during ascent
    if j > 0:
        v_rel_x = vx_launch[j] - wind
        v_rel_y = vy_launch[j] - 0  # no vertical wind
        v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)

        if v_rel_mag > 0.01:  # avoid division by zero

            # No drag for rocket ascending

            drag_magnitude = 0.5 * rho * rocketarea * streamlinecod * v_rel_mag**2
            xdrag = -drag_magnitude * v_rel_x / v_rel_mag
            ydrag = -drag_magnitude * v_rel_y / v_rel_mag
        else:
            xdrag = 0
            ydrag = 0
    else:
        xdrag = 0
        ydrag = 0

    # Net forces = thrust + drag - weight
    forcex_launch.append(xthrust + xdrag)
    forcey_launch.append(ythrust + ydrag - gforce)

    accx_launch.append(forcex_launch[j]/mass)
    accy_launch.append(forcey_launch[j]/mass)

    vx_launch.append(vx_launch[j] + accx_launch[j]*dt_launch)
    vy_launch.append(vy_launch[j] + accy_launch[j]*dt_launch)

    x_launch.append(x_launch[j] + vx_launch[j]*dt_launch)
    y_launch.append(y_launch[j] + vy_launch[j]*dt_launch)

    t_launch.append(t_launch[j] + dt_launch)
    j = j + 1

    if vy_launch[j] <= 0: # If reaches apogee
        break

######## RECOVERY
```

```
## SET INITIAL VARIABLES

t0 = 0      # starting time, s
dt = 0.01   # time step, s
height = y_launch[j] #apogee height, m

apogeeindex = j

dheight = 500 #deployment height, m
rho = 1.225 #air density, kg/m^3
deploy_time = 0.3 #parachute 2 deployment period
collide_time = 0.1 #rocket collision time, s

t_max = 6000 #maximum time
n_steps = int((t_max - t0) / dt) + 1

## Arrays to save simulation information
time = np.zeros(n_steps) #s
xposition = np.zeros(n_steps) #m
yposition = np.zeros(n_steps) #m

xvelocity = np.zeros(n_steps) #m/s
yvelocity = np.zeros(n_steps) #m/s

xdrag = np.zeros(n_steps) #N
ydrag = np.zeros(n_steps) #N

xacc = np.zeros(n_steps) #m/s^2
yacc = np.zeros(n_steps) #m/s^2

xinstantforce = np.zeros(n_steps) #N
yinstantforce = np.zeros(n_steps) #N
yinstantforce[0] = -gforce

yposition[0] = height #m
yvelocity[0] = vy_launch[j] #m/s

xposition[0] = x_launch[j] #m
xvelocity[0] = vx_launch[j] #m/s

time[0] = 0  #s

xdrag[0] = 0 #N
ydrag[0] = 0 #N

xacc[0] = 0 #m/s^2
yacc[0] = -9.81 #m/s^2

# PART 0.5: AFTER PARACHUTE 1 DEPLOYED

for i in range(1, n_steps):
    # Calculate relative velocity
    v_rel_x = xvelocity[i-1] - wind
    v_rel_y = yvelocity[i-1] - 0  # no vertical wind
    v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)

    # Calculate drag force components
    if v_rel_mag > 0.01:
        drag_magnitude = 0.5 * rho * cod1 * area1 * v_rel_mag**2
        xdrag[i] = -drag_magnitude * v_rel_x / v_rel_mag
        ydrag[i] = -drag_magnitude * v_rel_y / v_rel_mag
    else:
        xdrag[i] = 0
        ydrag[i] = 0

    # Net forces
    xinstantforce[i] = xdrag[i]
    yinstantforce[i] = ydrag[i] - gforce  # weight pulls down

    # Acceleration
    xacc[i] = xinstantforce[i] / mass
    yacc[i] = yinstantforce[i] / mass

    # Update velocities
```

```
        xvelocity[i] = xvelocity[i-1] + xacc[i] * dt
        yvelocity[i] = yvelocity[i-1] + yacc[i] * dt

        # Update positions
        xposition[i] = xposition[i-1] + xvelocity[i] * dt
        yposition[i] = yposition[i-1] + yvelocity[i] * dt

        time[i] = time[i-1] + dt

        if yposition[i] <= dheight:
            break

    # Parachute 2 Deployment Phase
    momentumix = mass * xvelocity[i]
    momentumiy = mass * yvelocity[i]

    # Terminal velocity targets
    xvel = wind  # horizontal velocity matches wind
    yvel = -termvel  # negative because falling down
    momentumfx = mass * xvel
    momentumfy = mass * yvel

    fx = (momentumfx - momentumix) / deploy_time
    fy = (momentumfy - momentumiy) / deploy_time
    fdeploy = math.sqrt(fx**2 + fy**2)
    chute_force[b] = fdeploy

    for j in range(i+1, n_steps):
            xvelocity[j] = xvel
            yvelocity[j] = yvel

            xposition[j] = xposition[j-1] + xvelocity[j] * dt
            yposition[j] = yposition[j-1] + yvelocity[j] * dt
            time[j] = time[j-1] + dt

            if yposition[j] <= 0:
                xposition[j] = xposition[j-1]
                yposition[j] = 0
                break

    # PART III: Ground Collision Phase
    momentum = mass * yvelocity[j]
    fcollide = abs(momentum / collide_time)
    collision_force[b] = fcollide
    b = b + 1

    if fcollide <= thresh_force and fdeploy <= thresh_force:
        fifteen.append(constant)

list_thresh = [3500 for _ in range(200)]

plt.plot(arr, chute_force, color="darkgoldenrod", lw=2, label="Chute Force")
plt.plot(arr, collision_force, color="saddlebrown", lw=2, label="Ground Force")
plt.plot(arr, list_thresh, color="red", linestyle="dotted", lw=2, label="Threshold Force")
plt.title("Cd * Surface Area vs. Two Forces", fontweight='bold')

low = fifteen[0]
high = fifteen[-1]

plt.vlines(low, 0, 4000, lw=1.5, linestyle="dashed", color="forestgreen")
plt.vlines(high, 0, 4000, lw=1.5, linestyle="dashed", color="forestgreen")
plt.axvspan(low, high, color='lightgreen', alpha=0.35)
plt.axvspan(high, 280, color='lightcoral', alpha=0.35)
plt.axvspan(0, low, color='lightcoral', alpha=0.35)

plt.text(90, 2000, 'Success: Landed \n Safely!',
         horizontalalignment='center', verticalalignment='top',
         fontsize=12.5, fontweight="bold", color='black')
plt.text(168, 3000, 'Chute 2 \n Deployment \n Failure',
         horizontalalignment='center', verticalalignment='top',
         fontsize=13, color='black')
plt.text(50, 1000, 'Ground Collision \n Damage',
         horizontalalignment='center', verticalalignment='top',
         fontsize=13, color='black')
```

```
l = f"Low = {low : 0.0f}"
h = f"High = {high : 0.0f}"

plt.xlim(15, 200)
plt.ylim(0, 4000)
plt.scatter(low, 3500, zorder=2, s=80, c="dodgerblue", label=l)
plt.scatter(high, 3500, zorder=2, s=80, c="dodgerblue", label=h)

print("================ Recommended Cd * Area Range ================")
print(f"Low = {low : 0.0f} | High = {high : 0.0f}")
print("============================================================")
plt.xlabel("Cd * Surface Area (m^2)", fontsize=12)
plt.ylabel("Force (N)", fontsize=12)

plt.legend()
plt.show()
```

```
================ Recommended Cd * Area Range ================
Low =  46 | High =  133
============================================================
```

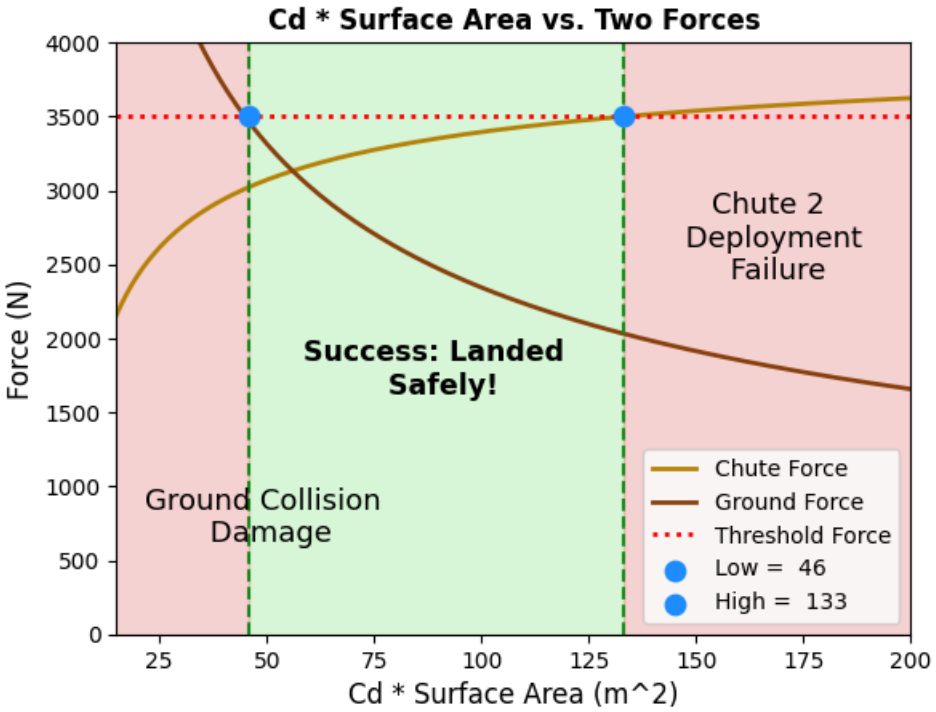


Figure 12. The range of constants $C = C_D \times A$ needed to create a successful rocket recovery is 46 to 133. The light brown line indicates the parachute 2 deployment force at different $C_d \times A$, and the darker brown indicates the ground collision force at different $C_d \times A$, where the dotted red line indicates the threshold force. The region of safe landing is then the region where both forces are less than 3500N. "Low" refers to the lowest constant $C_d \times A$ such that the two parameters yield a safe landing, and "High" refers to the highest constnat $C_d \times A$ such that the two parameters yield a safe landing. Values below "Low" lead to ground collision damage, whereas values above "High" lead to parachute 2 deployment failure.

Code Block Summary: This codeblock creates a graph showing the range of constants that create a safe rocket recovery. To do this, an array of constants are first created ranging from 15 to 200. Then, each of these constants are inputted into the previous code, calculating force, acceleration, velocity, and position at each time step. The parachute deployement and collision forces are then compared to the threshold force to determine if the given parachute constant will lead to a safe rocket recovery.

Alternatively, contour graphs displaying the coefficient of drag and surface area can be independently plotted as heat maps; while these graphs are harder to decipher, it provides a more specific understanding of the various combinations of coefficient of drag and cross-sectional areas that can be used to create a safe landing. The following code creates an overall rocket recovery outcome map, along with heat maps corresponding to parachute 2 deployment force and ground collision force, given a range of drag coefficients from 0.7 to 5, as well as surface areas from $5m^2$ to $200m^2$.

In [46]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap, BoundaryNorm

# storage arrays for outcomes and forces
cd_min, cd_max = 0.7, 4
area_min, area_max = 5.0, 100

cd_values = np.linspace(cd_min, cd_max, 100)
area_values = np.linspace(area_min, area_max, 100)
outcomes = np.zeros((100, 100))   # 0=success,1=ground damage,2=chute failure
chute_force_grid = np.zeros_like(outcomes)
collision_force_grid = np.zeros_like(outcomes)

def simulation(cod2, area2):

    ## CREATING EMPTY LISTS AND INITIAL VALUES FOR EACH VARIABLE
    rho = 1.225 #air density, kg/m^3
    mass = 70 # Rocket kg
    forcex_launch = []
    forcey_launch = []

    x_launch = []
    x_launch.append(0)

    y_launch = []
    y_launch.append(0)

    accx_launch = []
    accy_launch = []

    vx_launch = []
    vx_launch.append(0)

    vy_launch = []
    vy_launch.append(0)

    t_launch = []
    t_launch.append(0)
    dt_launch = 0.5
    gforce = mass * 9.81 #N, gravitational force

    angle = 90

    # Fixed Initial properties: Parachute 1
    cod1 = 0.7 #coefficient of drag
    area1 = 5 #cross-sectional surface area, m^2

    # Variable Properties: Parachute 2
    termvel = np.sqrt(2*mass*9.81/(rho*area2*cod2))

    rocketarea = 0 #for drag during ascent
    streamlinecod = 0

    ## PART 0: ASCENT TO APOGEE

    j = 0
    while True:
        # Thrust forces
        if t_launch[j] <= 5:
            xthrust = thrust_force * math.cos(math.radians(angle))
            ythrust = thrust_force * math.sin(math.radians(angle))
        else:
            xthrust = 0
            ythrust = 0

        # Drag forces during ascent
        if j > 0:
            v_rel_x = vx_launch[j] - wind
            v_rel_y = vy_launch[j] - 0  # no vertical wind
            v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)

            if v_rel_mag > 0.01:  # avoid division by zero
```

```
            # No drag for rocket ascending

            drag_magnitude = 0.5 * rho * rocketarea * streamlinecod * v_rel_mag**2
            xdrag = -drag_magnitude * v_rel_x / v_rel_mag
            ydrag = -drag_magnitude * v_rel_y / v_rel_mag
        else:
            xdrag = 0
            ydrag = 0
    else:
        xdrag = 0
        ydrag = 0

    # Net forces = thrust + drag - weight
    forcex_launch.append(xthrust + xdrag)
    forcey_launch.append(ythrust + ydrag - gforce)

    accx_launch.append(forcex_launch[j]/mass)
    accy_launch.append(forcey_launch[j]/mass)

    vx_launch.append(vx_launch[j] + accx_launch[j]*dt_launch)
    vy_launch.append(vy_launch[j] + accy_launch[j]*dt_launch)

    x_launch.append(x_launch[j] + vx_launch[j]*dt_launch)
    y_launch.append(y_launch[j] + vy_launch[j]*dt_launch)

    t_launch.append(t_launch[j] + dt_launch)
    j = j + 1

    if vy_launch[j] <= 0: # If reaches apogee
        break

######## RECOVERY

## SET INITIAL VARIABLES

t0 = 0      # starting time, s
dt = 0.5   # time step, s
height = y_launch[j] #apogee height, m

dheight = 500 #deployment height, m
rho = 1.225 #air density, kg/m^3
deploy_time = 0.3 #parachute 2 deployment period
collide_time = 0.1 #rocket collision time, s

t_max = 6000 #maximum time
n_steps = int((t_max - t0) / dt) + 1

## Arrays to save simulation information
time = np.zeros(n_steps) #s
xposition = np.zeros(n_steps) #m
yposition = np.zeros(n_steps) #m

xvelocity = np.zeros(n_steps) #m/s
yvelocity = np.zeros(n_steps) #m/s

xdrag = np.zeros(n_steps) #N
ydrag = np.zeros(n_steps) #N

xacc = np.zeros(n_steps) #m/s^2
yacc = np.zeros(n_steps) #m/s^2

xinstantforce = np.zeros(n_steps) #N
yinstantforce = np.zeros(n_steps) #N
yinstantforce[0] = -gforce

yposition[0] = height #m
yvelocity[0] = vy_launch[j] #m/s

xposition[0] = x_launch[j] #m
xvelocity[0] = vx_launch[j] #m/s

time[0] = 0  #s

xdrag[0] = 0 #N
```

```
ydrag[0] = 0 #N

xacc[0] = 0 #m/s^2
yacc[0] = -9.81 #m/s^2

# PART 0.5: AFTER PARACHUTE 1 DEPLOYED

for i in range(1, n_steps):
    # Calculate relative velocity
    v_rel_x = xvelocity[i-1] - wind
    v_rel_y = yvelocity[i-1] - 0  # no vertical wind
    v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)

    # Calculate drag force components
    if v_rel_mag > 0.01:
        drag_magnitude = 0.5 * rho * cod1 * area1 * v_rel_mag**2
        xdrag[i] = -drag_magnitude * v_rel_x / v_rel_mag
        ydrag[i] = -drag_magnitude * v_rel_y / v_rel_mag
    else:
        xdrag[i] = 0
        ydrag[i] = 0

    # Net forces
    xinstantforce[i] = xdrag[i]
    yinstantforce[i] = ydrag[i] - gforce  # weight pulls down

    # Acceleration
    xacc[i] = xinstantforce[i] / mass
    yacc[i] = yinstantforce[i] / mass

    # Update velocities
    xvelocity[i] = xvelocity[i-1] + xacc[i] * dt
    yvelocity[i] = yvelocity[i-1] + yacc[i] * dt

    # Update positions
    xposition[i] = xposition[i-1] + xvelocity[i] * dt
    yposition[i] = yposition[i-1] + yvelocity[i] * dt

    time[i] = time[i-1] + dt

    if yposition[i] <= dheight:
        break

# Parachute 2 Deployment Phase
momentumix = mass * xvelocity[i]
momentumiy = mass * yvelocity[i]

# Terminal velocity targets
xvel = wind  # horizontal velocity matches wind
yvel = -termvel  # negative because falling down
momentumfx = mass * xvel
momentumfy = mass * yvel

fx = (momentumfx - momentumix) / deploy_time
fy = (momentumfy - momentumiy) / deploy_time
fdeploy = math.sqrt(fx**2 + fy**2)

for j in range(i+1, n_steps):
    xvelocity[j] = xvel
    yvelocity[j] = yvel

    xposition[j] = xposition[j-1] + xvelocity[j] * dt
    yposition[j] = yposition[j-1] + yvelocity[j] * dt
    time[j] = time[j-1] + dt

    if yposition[j] <= 0:
        xposition[j] = xposition[j-1]
        yposition[j] = 0
        break

# PART III: Ground Collision Phase
momentum = mass * yvelocity[j]
fcollide = abs(momentum / collide_time)
```

```
    if fdeploy > thresh_force:
        outcome = 2
    elif fcollide > thresh_force:
        outcome = 1
    else:
        outcome = 0

    return fdeploy, fcollide, outcome
```

Code Block: This code block simulates the rocket's descent, calculating force, acceleration, velocity, and position. The same code as before was used but is packaged into a function to use later.

In [47]:
```
# Find all forces and outcomes

for i_cd, cd in enumerate(cd_values):

    for i_a, area in enumerate(area_values):
        f_deploy, f_collide, code = simulation(cd, area)
        chute_force_grid[i_cd, i_a] = f_deploy
        collision_force_grid[i_cd, i_a] = f_collide
        outcomes[i_cd, i_a] = code


# Plotting contour map

Cd_mesh, Area_mesh = np.meshgrid(cd_values, area_values, indexing='ij')

fig, ax = plt.subplots(figsize=(8, 5))

cmap = ListedColormap(["lightgreen", "lightcoral", "gold"])
bounds = [-0.5, 0.5, 1.5, 2.5]
norm = BoundaryNorm(bounds, cmap.N)

cf = ax.contourf(Area_mesh, Cd_mesh, outcomes,
                 levels=[-0.5,0.5,1.5,2.5],
                 colors=["lightgreen","lightcoral","gold"],
                 alpha=0.9)

cbar = fig.colorbar(cf, ticks=[0, 1, 2], aspect=30)
cbar.ax.set_yticklabels(['Success (safe)', 'Ground collision damage', 'Chute 2 failure'])
ax.set_xlabel("Parachute 2 Surface Area (m²)", fontsize=14)
ax.set_ylabel("Parachute 2 Coefficient of Drag (Cd)", fontsize=14)
ax.set_title("Rocket Recovery Outcome Map", fontsize=15, fontweight = "bold")

for const in [15, 50, 100, 150, 200]:
    area_curve = const / cd_values
    mask = (area_curve >= area_min) & (area_curve <= area_max)
    ax.plot(area_curve[mask], cd_values[mask], color='k',
            linewidth=0.7, linestyle='--', alpha=0.6)

import matplotlib.patches as mpatches
p0 = mpatches.Patch(color="lightgreen", label="Success: Safe landing")
p1 = mpatches.Patch(color="lightcoral", label="Ground collision damage")
p2 = mpatches.Patch(color="gold", label="Chute 2 deployment failure")
ax.legend(handles=[p0, p1, p2], loc='upper right')

plt.tight_layout()
plt.show()

# Optional: force visualizations
fig2, (ax1, ax2) = plt.subplots(1, 2, figsize=(15,5))

cf1 = ax1.contourf(Area_mesh, Cd_mesh, chute_force_grid, cmap='viridis')
ax1.set_title('Parachute 2 Deployment Force (N)', fontweight = "bold", fontsize = 15)
ax1.set_xlabel('Area (m²)', fontsize = 14)
ax1.set_ylabel('Coefficient of Drag (Cd)', fontsize = 14)

cf2 = ax2.contourf(Area_mesh, Cd_mesh, collision_force_grid, cmap='viridis')
ax2.set_title('Ground Collision force (N)', fontweight = "bold", fontsize = 15)
ax2.set_xlabel('Area (m²)', fontsize = 14)
ax2.set_ylabel('Coefficient of Drag (CD)', fontsize = 14)

cbar1 = fig2.colorbar(cf1, ax=ax1)
```

```
cbar1.set_label('Parachute 2 Deployment Force (N)', fontsize=13)

cbar2 = fig2.colorbar(cf2, ax=ax2)
cbar2.set_label('Ground Collision force (N)', fontsize=13)

plt.tight_layout()
plt.show()
```

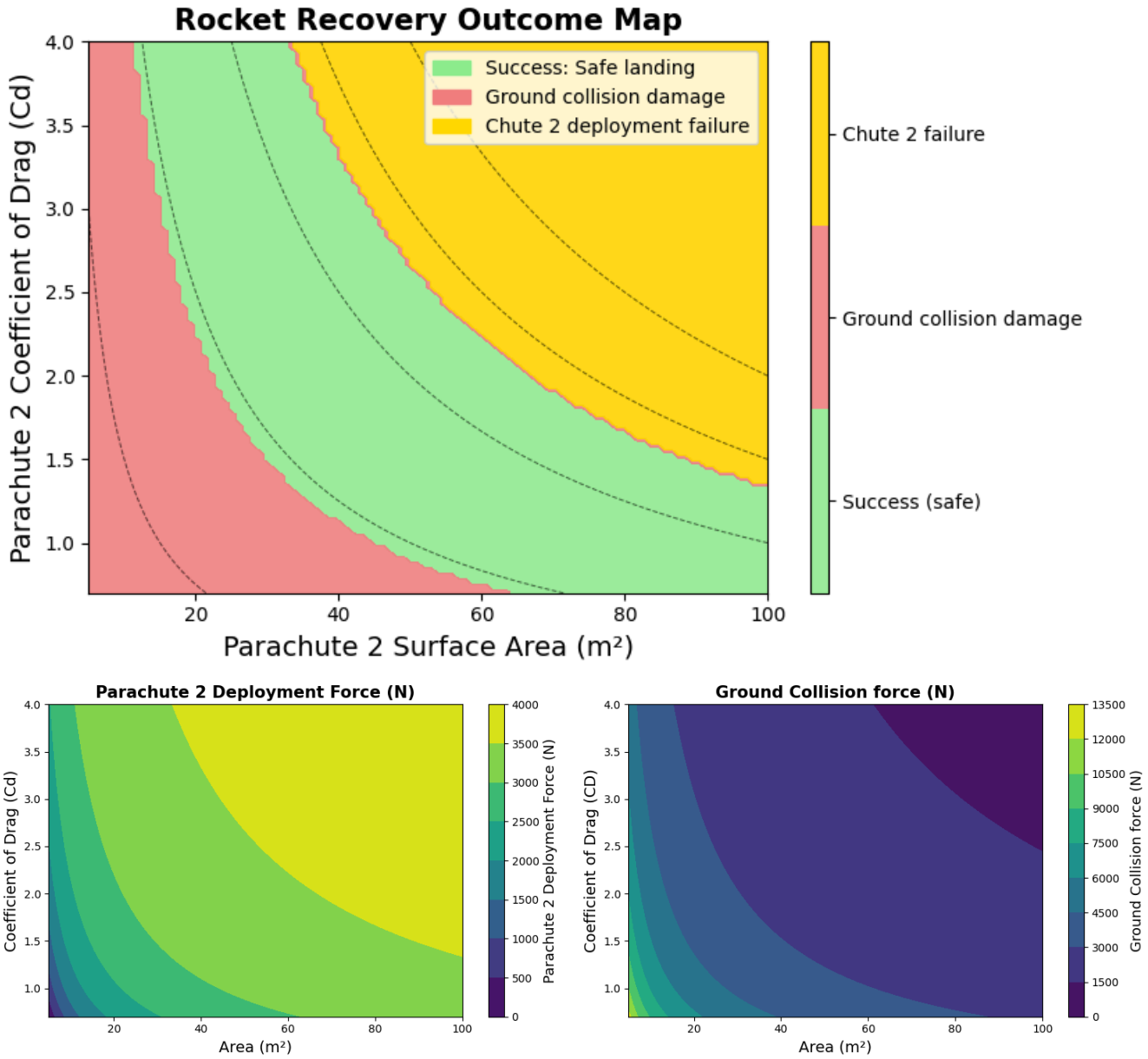


Figure 13a,b,c. Area and coefficient of drag are proportional to Parachute 2 Deployment Force (13a), while area and coefficient of drag are inversely proportional to Ground Collision Force (13b). So, regions with low constants $C = C_d \times A$ correspond to Ground collision damage, while regions with high constants $C = C_d \times A$ correspond to Chute 2 failure.

Code Block Summary: The function simulation() is used in this codeblock to calculate the rocket's outcome, deployment force, and collision force for each pair of drag coefficients and surface areas in the specified range. Matplotlib is then used to generate heat maps corresponding to these results.

Step 1 Analysis

From **Figure 12.**, the possible range of constant $C = C_D \times A$ is 46-133. If the constant is higher, the rocket will suffer from Parachute 2 deployment failure, where the Parachute 2 deployment force exceeds 3500N. If the constant is lower, the rocket will suffer from ground collision damage, where the ground collision force exceeds 3500N.

From **Figure 13a,b,c**, more detailed plots show the region of the coefficients of drag and surface areas where the rocket will produce a safe landing, along with heat maps displaying Parachute 2 Deployment Force and Ground Collision Force as a function of drag coefficient and surface area. All three heat maps show exponential-like curves, which can be interpreted

as the interactions between the coefficient of drag and surface area. From the heat maps of the individual forces, surface area and coefficient of drag are proportional to parachute deployment force, and inversely proportional to ground collision force. Put together, regions where drag coefficients and surface areas are high (Large constant C) correspond to Chute 2 failure, and regions where drag coefficients and surface areas are low (Small constant C) correspond to Ground collision damage, as seen in the Rocket Recovery Outcome Map. This aligns with all other analyses conducted in this project.

3.2 Step 2: Range of Parameters Creating a Successful Rocket Recovery

Finally, we take the range found from Step 1 of the quantitative phase space scan and find the subset initial angles within that range needed to create a successful rocket recovery. For this, we take the constant $C = C_D \times A$ and vary θ to be between $80^o \leq \theta \leq 135^o$; this range we decided upon after multiple preliminary trials.

In [48]:
```
# Storage arrays for outcomes and forces
constant_min, constant_max = 46,133
angle_min, angle_max = 80, 115

const_values = np.linspace(constant_min, constant_max, 100)
angle_values = np.linspace(angle_min, angle_max, 100)

position = np.zeros((100, 100))

def sim_position(const, angle):
    good = []

    ## CREATING EMPTY LISTS AND INITIAL VALUES FOR EACH VARIABLE
    rho = 1.225 #air density, kg/m^3
    mass = 70 # Rocket kg
    forcex_launch = []
    forcey_launch = []

    x_launch = []
    x_launch.append(0)

    y_launch = []
    y_launch.append(0)

    accx_launch = []
    accy_launch = []

    vx_launch = []
    vx_launch.append(0)

    vy_launch = []
    vy_launch.append(0)

    t_launch = []
    t_launch.append(0)
    dt_launch = 0.5
    gforce = mass * 9.81 #N, gravitational force

    # Fixed Initial properties: Parachute 1
    cod1 = 0.7 #coefficient of drag
    area1 = 5 #cross-sectional surface area, m^2

    # Variable Properties: Parachute 2
    termvel = np.sqrt(2*mass*9.81/(rho*const))

    rocketarea = 0 #for drag during ascent
    streamlinecod = 0

    ## PART 0: ASCENT TO APOGEE

    j = 0
    while True:
        # Thrust forces
        if t_launch[j] <= 5:
            xthrust = thrust_force * math.cos(math.radians(angle))
            ythrust = thrust_force * math.sin(math.radians(angle))
        else:
            xthrust = 0
            ythrust = 0
```

```
    # Drag forces during ascent
    if j > 0:
        v_rel_x = vx_launch[j] - wind
        v_rel_y = vy_launch[j] - 0  # no vertical wind
        v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)

        if v_rel_mag > 0.01:  # avoid division by zero

            # No drag for rocket ascending

            drag_magnitude = 0.5 * rho * rocketarea * streamlinecod * v_rel_mag**2
            xdrag = -drag_magnitude * v_rel_x / v_rel_mag
            ydrag = -drag_magnitude * v_rel_y / v_rel_mag
        else:
            xdrag = 0
            ydrag = 0
    else:
        xdrag = 0
        ydrag = 0

    # Net forces = thrust + drag - weight
    forcex_launch.append(xthrust + xdrag)
    forcey_launch.append(ythrust + ydrag - gforce)

    accx_launch.append(forcex_launch[j]/mass)
    accy_launch.append(forcey_launch[j]/mass)

    vx_launch.append(vx_launch[j] + accx_launch[j]*dt_launch)
    vy_launch.append(vy_launch[j] + accy_launch[j]*dt_launch)

    x_launch.append(x_launch[j] + vx_launch[j]*dt_launch)
    y_launch.append(y_launch[j] + vy_launch[j]*dt_launch)

    t_launch.append(t_launch[j] + dt_launch)
    j = j + 1

    if vy_launch[j] <= 0: # If reaches apogee
        break

######## RECOVERY

## SET INITIAL VARIABLES

t0 = 0      # starting time, s
dt = 0.5   # time step, s
height = y_launch[j] #apogee height, m

dheight = 500 #deployment height, m
rho = 1.225 #air density, kg/m^3

t_max = 6000 #maximum time
n_steps = int((t_max - t0) / dt) + 1

## Arrays to save simulation information
time = np.zeros(n_steps) #s
xposition = np.zeros(n_steps) #m
yposition = np.zeros(n_steps) #m

xvelocity = np.zeros(n_steps) #m/s
yvelocity = np.zeros(n_steps) #m/s

xdrag = np.zeros(n_steps) #N
ydrag = np.zeros(n_steps) #N

xacc = np.zeros(n_steps) #m/s^2
yacc = np.zeros(n_steps) #m/s^2

xinstantforce = np.zeros(n_steps) #N
yinstantforce = np.zeros(n_steps) #N
yinstantforce[0] = -gforce

yposition[0] = height #m
yvelocity[0] = vy_launch[j] #m/s
```

```
    xposition[0] = x_launch[j] #m
    xvelocity[0] = vx_launch[j] #m/s

    time[0] = 0  #s

    xdrag[0] = 0 #N
    ydrag[0] = 0 #N

    xacc[0] = 0 #m/s^2
    yacc[0] = -9.81 #m/s^2

    # PART 0.5: AFTER PARACHUTE 1 DEPLOYED
    # Terminal velocity targets
    xvel = wind
    yvel = -termvel

    for i in range(1, n_steps):
        # Calculate relative velocity
        v_rel_x = xvelocity[i-1] - wind
        v_rel_y = yvelocity[i-1]
        v_rel_mag = math.sqrt(v_rel_x**2 + v_rel_y**2)

        # Calculate drag force components
        if v_rel_mag > 0.01:
            drag_magnitude = 0.5 * rho * cod1 * area1 * v_rel_mag**2
            xdrag[i] = -drag_magnitude * v_rel_x / v_rel_mag
            ydrag[i] = -drag_magnitude * v_rel_y / v_rel_mag
        else:
            xdrag[i] = 0
            ydrag[i] = 0

        # Net forces
        xinstantforce[i] = xdrag[i]
        yinstantforce[i] = ydrag[i] - gforce  # weight pulls down

        # Acceleration
        xacc[i] = xinstantforce[i] / mass
        yacc[i] = yinstantforce[i] / mass

        # Update velocities
        xvelocity[i] = xvelocity[i-1] + xacc[i] * dt
        yvelocity[i] = yvelocity[i-1] + yacc[i] * dt

        # Update positions
        xposition[i] = xposition[i-1] + xvelocity[i] * dt
        yposition[i] = yposition[i-1] + yvelocity[i] * dt

        time[i] = time[i-1] + dt

        if yposition[i] <= dheight:
            break


    for j in range(i+1, n_steps):
        xvelocity[j] = xvel
        yvelocity[j] = yvel

        xposition[j] = xposition[j-1] + xvelocity[j] * dt
        yposition[j] = yposition[j-1] + yvelocity[j] * dt
        time[j] = time[j-1] + dt

        if yposition[j] <= 0:
            xposition[j] = xposition[j-1]
            yposition[j] = 0
            break

    # PART III: Ground Collision Phase

    return abs(xposition[j])
```

In [50]:

```
#======================== FIND ALL POSITIONS ========================#
print("================PROGRESS UPDATE================ \n")
```

```
position = np.zeros((len(const_values), len(angle_values)))
good = []
for i_c, const in enumerate(const_values):

    for i_a, angle in enumerate(angle_values):
        final_x = sim_position(const, angle)
        if final_x <= 100:
            good.append((const, angle, final_x))
        position[i_c, i_a] = final_x

#======================== PLOTTING CONTOUR MAP ========================#
Const_mesh, Angle_mesh = np.meshgrid(const_values, angle_values, indexing='ij')
fig, ax = plt.subplots(figsize=(8, 5))
cf = ax.pcolormesh(Angle_mesh, Const_mesh, position, shading='auto', cmap='plasma')

# Plot continuous heat map
fig.colorbar(cf, ax=ax, label='Landing Distance from Launch (m)')
ax.set_xlabel("Parachute 2 Deployment Angle (degrees)", fontsize=14)
ax.set_ylabel("Parachute 2 Cd * Surface Area (m²)", fontsize=14)
ax.set_title("Landing Distance from Launch vs. Deployment Angle and Cd * Area", fontsize=15, fontweight
plt.tight_layout()
plt.show()
```

```
===============PROGRESS UPDATE===============
```

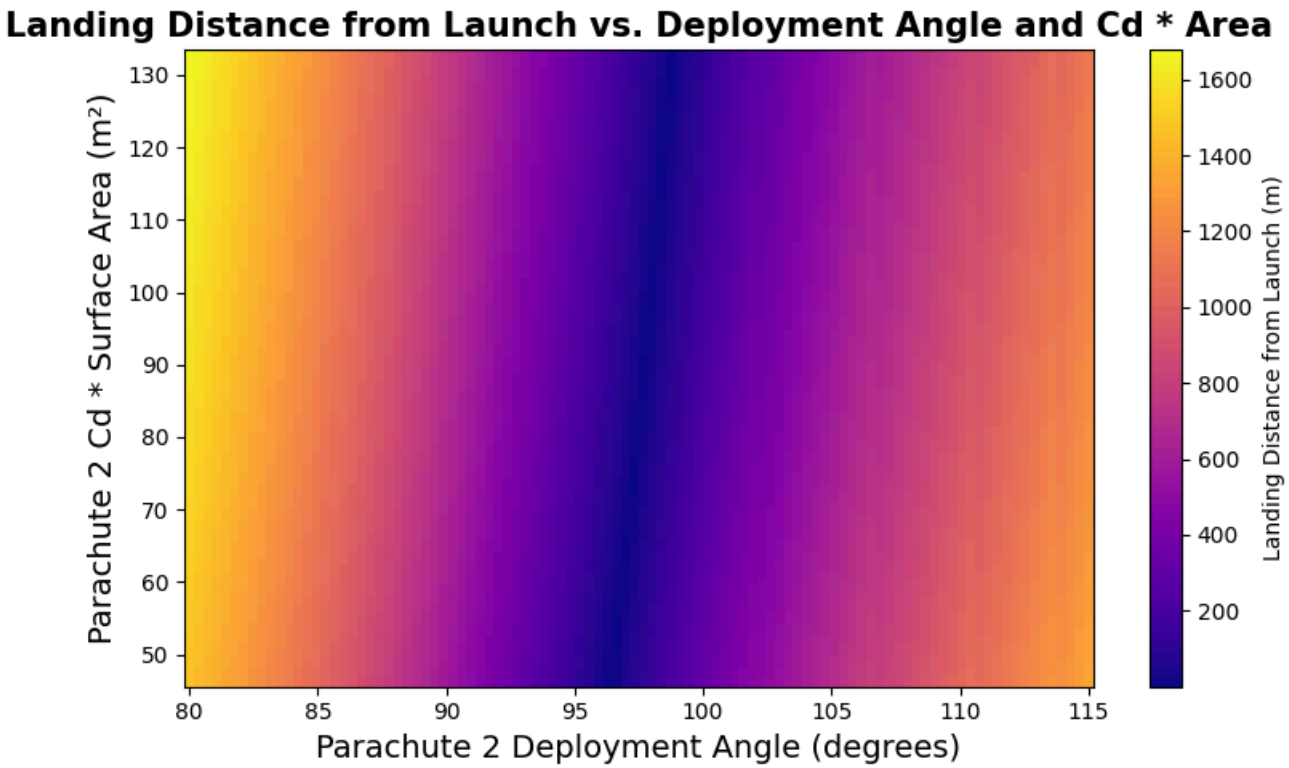


Figure 14. Heat map of the final rocket distance from original launching position. The landing distance from the launch is lowest along the centre diagonal purple line, whereas the landing distance form the launch is highest along the outer diagonal yellow and orange lines.

In [51]:
```
# Plot recommended values as scatter graph
fig, ax = plt.subplots(figsize=(8, 5))
for val in good:
    ax.scatter(val[1], val[0], color='blue', s=10)
ax.set_xlabel("Parachute 2 Deployment Angle (degrees)", fontsize=14)
ax.set_ylabel("Parachute 2 Cd * Surface Area (m²)", fontsize=14)
ax.set_title("Recommended Parachute 2 Deployment Angle and Cd * Area \n for Landing within 100m of Launc
ax.set_xlim(80, 115)
ax.set_ylim(46, 133)

# Plot left line with slope and intercept
import numpy as np
x = np.array([80, 115])
slope = 40
intercept = -3750
```

```
y = slope * (x-0.6) + intercept
ax.plot(x, y, color='red', linestyle='--', label='Left Boundary Line', linewidth=2)
ax.legend()

# Plot right line with slope and intercept

x = np.array([80, 115])
slope = 40
intercept = -3750
y = slope * (x-2.9) + intercept
ax.plot(x, y, color='green', linestyle='--', label='Right Boundary Line', linewidth=2)
ax.legend()

# Find minimum initial launching angle from good
constants = [item[0] for item in good]
angles = [item[1] for item in good]
print("==============SIMULATION RESULTS================ \n")
print(f"Reasonable range of angles: {min(angles): 0.1f}-{max(angles):0.1f}\n")
print(f"Within this range of angles, constant C = Cd x A must fall within the parallelogram defined by:


plt.show()
```

```
==============SIMULATION RESULTS================

Reasonable range of angles:  95.6-99.8

Within this range of angles, constant C = Cd x A must fall within the parallelogram defined by:
 * Cd =  46
 * Cd =  133
 * Cd = 40(θ-2.9)-3750
 * Cd = 40(θ-0.6)-3750
```

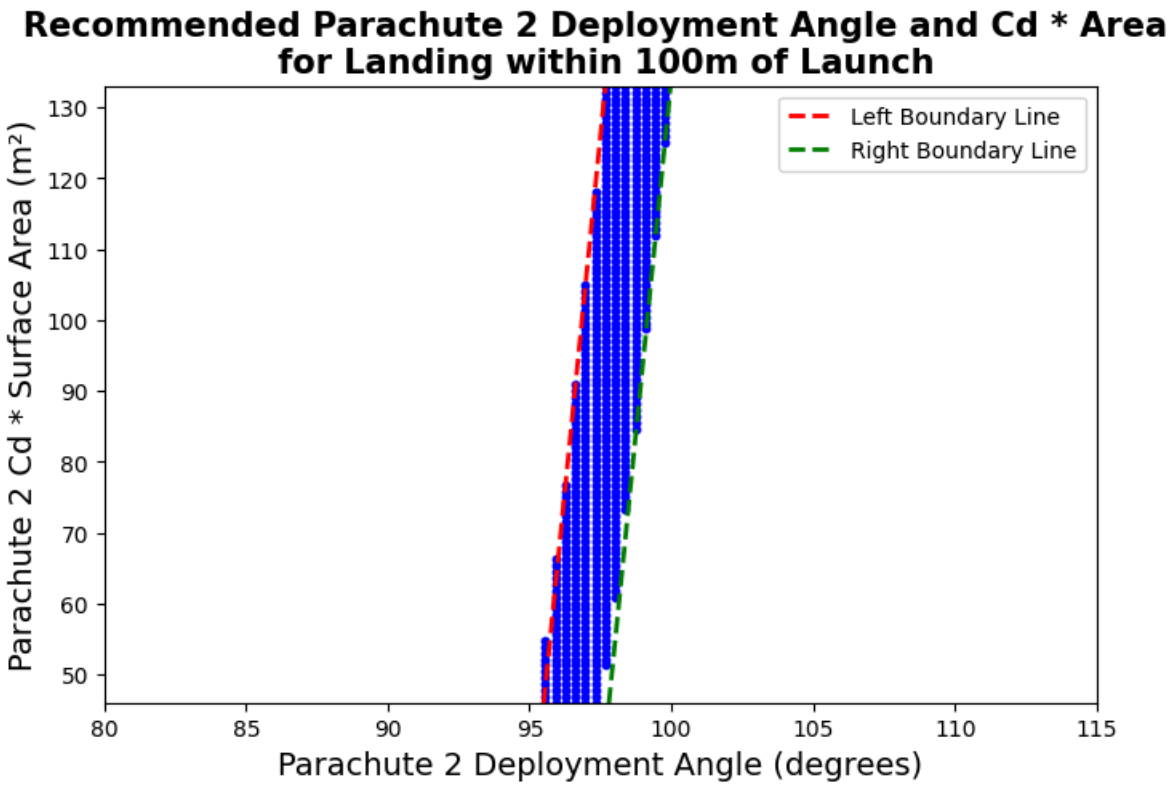


Figure 15. Graph of all combinations of $C = C_D \times A$ and deployment angle where final rocket position is 100m fron original launching position. The valid values create a parallelogram defined by $C_d = 45, C_d = 133, C_d = 40(\theta - 2.9) - 3750, C_d = 40(\theta - 0.6) - 3750.$

3.3 Quantitative Phase Space Analysis

From our quantitative phase space analysis, we see that a diagonal of constants $C = C_d \times A$ and Parachute 2 Deployment angle lead to a successful rocket recovery. We can further quantify this using linear equations.

From **Figure 14**, the approximate left boundary line and right boundary line is drawn. We can also find the minimum and maximum values of of deployment angles that fall under the blue diagonal line in Figure 14. Then, we can say that the region of possible combinations of drag coefficient, surface area, and deployment angle is a parallelogram defined by the following restrictions:

- $95.6^o \leq \theta \leq 99.8^o$
- $46 \leq C \leq 133$
- For some θ where $95.6^o \leq \theta \leq 99.8^o$, $C \leq 40(\theta - 2.9) - 3750$
- For some θ where $95.6^o \leq \theta \leq 99.8^o$, $C \geq 40(\theta - 0.6) - 3750$

4. Results & Discussion

4.1 Summary of Findings

From Preliminary Trials with Wind

- **Drag Coefficient and Surface Area Too Small**: When coefficient of drag and surface area are too small, the ground collision force experienced by the rocket is very large, leading to a failed landing.
- **Drag Coefficient and Surface Area Too Large**: Conversly, when coefficient of drag and surface area are too large, the deployment force experienced by the rocket is very lerge, also leading to a failed landing.
- **Finding the middle**: To find combinations of these two parameters where the rocket lands safely, one needs to find the middle of these two extremes, where drag coefficient and area are enough to slow the rocket down before landing while not creating excess force on the rocket when parachute 2 is deployed.
- **Deployment Angle & Apogee**: As deployment angle gets close to 90 degrees, the rocket's apogee increases due to the larger vertical velocity coponent.
- **Deployment Angles & Force**: Deployment angle does not affect collision and deployment force, given that (1) the angle within the range at which parachute 1 terminal velocity is still achieved before parachute 2 deployment, and (2) the rocket's apogee is higher than parachute 2's deployment height.

From Quantitative Phase Space Analysis

- The range of constants

$$C = C_D \times A$$

  where both the parachute 2 deployment and ground collision force are under 3500N is **46-133**.

- The constants C and deployment angle θ must satisfy the following conditions for the rocket to achieve a successful landing:

  1. $95.6^o \leq \theta \leq 99.8^o$
  2. $46 \leq C \leq 133$
  3. For some θ where $95.6^o \leq \theta \leq 99.8^o$, $C \leq 40(\theta - 2.9) - 3750$
  4. For some θ where $95.6^o \leq \theta \leq 99.8^o$, $C \geq 40(\theta - 0.6) - 3750$

4.2 Limitations & Next Steps

4.2.1 Technical Limitations

- **Machine Precision**: Since we used Euler's method in this project, we have difficulties in machine precision– we are approximating the velocity and position by *discretizing* our calculations, reducing the accuracy of our simulation.
- **CPU Memory & Simplistic Model**: Since we are running this project on a limited CPU memory, it is not feasible to develop overly-complicated simulations that accurately model real-world scenarios; the number of external factors we would need to account for would skyrocket (ha!) the complexity of our simulations, such that the CPU will not be able to run the programs.

4.2.2 Application Limitations

- **External Validity**: In this project, multiple physical assumptions were made about the launch environment, rocket, and two parachutes, including a constant air density, no energy loss other than drag, massless parachutes, and a point-like rocket shape. This limits the project's external validity and application to real-world scenarios.
- **3-D Trajectory**: In this project, the rocket is assumed to only have a vertical and horizontal trajectory. While this is often true for rocket launches, this generalization discounts the possible effects of random angles and other effects.
- **Lack of Uncertainty Calculations**: The lack of uncertainty calculations in the rocket's physical properties and vertical trajectory can limit the accuracy of this project's results.

4.2.3 Next Steps

- **Incorporating Random Angles in Launches**: Instead of simply simulating a vertical rocket trajectory, incoroprating random angle changes during the rocket's ascent and descent would be more representative of real-world scenarios, where wind and other forces are also present. This would incorporate nuances in the rocket's horizontal velocity and and experienced drag force component that would affect the rocket's landing position.
- **Incorporating multiple parachutes**: More parachutes can be added into the simulation, where the objective then becomes solving how many chutes is needed for a rocket to land under a threshold force.

5. Acknowledgements

(This section is where you acknowledge any help or support that you received in completing your project. It must include a statement regarding if and how Generative AI or other help was used in completing your project.)

I want to acknowledge Joss and the PHYS 210 TAs who supported me throughout this project; this ranged from helping me determine which assumptions I should incorporate into my project, and how I can graphically represent my simulation in an effective way. I also want to acknowledge Sumaiya Hossain and Sam Quastel for providing suggestions in the preliminary stages of my project, including what additional physics to use.

Generative AI was used for preliminary research into rocket recovery force ranges, collision durations, and thrust force estimates. ChatGPT was also used to help format the contour maps created in the analysis portion of the project.

6. References

(In addition to citations for any python packages used beyond our standard ones, list the sources for any data or literature cited in your project. Additionally, you must also cite the sources for any code that you found on the internet or from peers.)

[1] https://www.grc.nasa.gov/www/k-12/VirtualAero/BottleRocket/airplane/rktrecv.html

[2] https://www.grc.nasa.gov/www/k-12/VirtualAero/BottleRocket/airplane/termv.html

[3] https://www.grc.nasa.gov/www/k-12/VirtualAero/BottleRocket/airplane/rktvrecv.html

[4] https://wikis.mit.edu/confluence/display/RocketTeam/Basic+Recovery+CONOPS

[5] https://www.apogeerockets.com/Intro-to-Dual-Deployment

[6] https://en.wikipedia.org/wiki/G-force

[7] https://www.ubcrocket.com/past-projects.html#:~:text=The%20Co%2DPilot%20project%20was,and%20advances%20in%20heatsink%20frameworks.

[8] https://www.youtube.com/watch?v=Lq0JWGOdtnc

[9] https://en.wikipedia.org/wiki/Model_rocket_motor_classification

[10] https://www.youtube.com/watch?v=Q69AoYo9j1M

[11] https://www.valkyrierecoverysystems.com/uploads/1/1/9/5/119545746/hobby_parachute_cd_master_list.pdf?srsltid=AfmBOoq2Ki1LffyIYwlCxjBu88Eo8_CzQAuaHtMqbaUrg4eiboz04ZSn

[12] https://www.apogeerockets.com/education/downloads/Newsletter361.pdf

[13] https://www.youtube.com/watch?v=l9Ap8vnd2dQ&t=47s

[14] https://www.researchgate.net/publication/316618143_Performance_of_RC_Beams_with_or_without_FRP_Strengthening_Subje

7. Appendix 1: Code validation

A1.1: Comparison of simulation results with by-hand calculations

In this validation tasks, we check the 1-Dimensional simulation introduced in portion (1) of analysis. We choose to use the 1-Dimensional simulation, since this is the basic code we use to extend to 2-Dimensions, and ensuring that the 1-Dimensional simulation code works is crucial for the rest of our analysis.

For the first validation task, I check the 1-Dimensional simulation's calculated Parachute 2 deployment forces between Attempts 1 and 2. In my hand calculations, I assume that the rocket reaches Parachute 1's terminal velocity, which is a valid assumption given the late time Parachute 2 is deployed. I then calculate Parachute 2's terminal velocity for each attempt, and use the formula

$$F = \frac{\Delta p}{\Delta t} = \frac{m(v_{1term} - v_{2term})}{0.3}$$

t0 calculate the force.

These by-hand calculations only differ from the simulation force by 2N each, indicating that the behaviour of my simulation is expected:

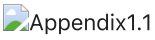


Figure A.1. Image of hand calculations of Parachute 2 deployment forces.

A1.2:

For the second validation task, we check the velocity and positional trajectory of the rocket in Attempts 2 and 3 of the preliminary trials in our 1-D simulation.

Our graphs should be separated into four main parts: when (1) Parachute 1 is deployed and its terminal velocity is reached, (2) Parachute 2 is deployed and the rocket's velocity immediately drops to its terminal velocity, (3) the rocket descends down to ground at constant Parachute 2 terminal velocity, and (4) the rocket collides with the ground, and velocity promptly becomes zero.

We analyze these four parts separately:

1. **Parachute 1 deployment**: After parachute 1 is deployed, we should see a sudden increase in velocity; however, the rocket's acceleration should decrease over time to zero as Parachute 1's terminal velocity is reached. We see this in the left portions of Attempt 2 and 3's graphs, where velocity increases and eventually levels out. This is also seen in the rocket's Position vs. Time graphs, where the slope of the rocket's position decreases until it reachs a constant value at terminal velocity (note that the Velocity vs. Time graph indicates velocity as positive, whereas terminal velocity indicates velocity as negative. This was done to increase user understandability, since downwards velocity and positions are often modelled as increasing and decreasing, respectively).
2. **Parachute 2 deployment**: When parachute 2 is deployed, we should see a sudden drop in velocity, since the rocket's terminal velocity decreases to Parachute 2's. This is what we see in both Attempts 2 and 3 graphs.
3. **Rocket Descent**: After parachute 2, the graphs indicate a constant descent to the ground, which is was we expect, since the rocket is moving at Parachute 2's terminal velocity.
4. **Rocket Collision with the ground**: When the rocket collides with the ground, the velocity graph should indicate a sudden decrease to $0 m/s$, and the position should indicate 0. This is seen across both Attempts 2 and 3.

Figure 16. Comparisons of different rocket recovery graphs.

8. Appendix 2: Sensitivity Analysis

In this appendix, the sensitivity of the 1-Dimensional simulation is tested by slightly varying the parachute 2 deployment time. In Trial 1, the parachute 2 deployment time is set to be 0.3s, whereas in Trial 2, the parachute 2 deployment time is set to be 0.4s.

Trial 1: Parachute Deployment Time = $0.3s$

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

## USER INPUTS_THRESHOLD FORCE AND THRUST FORCE
thrust_force = 1000
Thresh_force = 3500

##### LAUNCHING

## CREATING EMPTY LISTS AND INITIAL VALUES FOR EACH VARIABLE
mass = 70 # Rocket kg
force_launch = []

height_launch = []
height_launch.append(0)

acceleration_launch = []

velocity_launch = []
velocity_launch.append(0)

t_launch = []
t_launch.append(0)
dt_launch = 0.1


##########
```

Code Summary: This code block creates empty lists and initial values for the needed variables for the simulation, including force, height, acceleration, velocity, and time. The rocket's thrust force and threshold force of 3500 are also defined.

In [ ]:
```
## PART 0: ASCENT TO APOGEE

#### Modeling ascent using Euler's Method with while loop
j = 0
while True:
    if t_launch[j] <= 5:
        force_launch.append(thrust_force - mass * 9.8)
    else:
        force_launch.append(-mass*9.8)
    acc = force_launch[j]/mass
    acceleration_launch.append(acc)
    velocity_launch.append(velocity_launch[j] + acceleration_launch[j])
    height_launch.append(height_launch[j] + velocity_launch[j] * dt_launch)
    t_launch.append(t_launch[j] + dt_launch)

    j = j + 1
    if velocity_launch[j] <= 0: # If reaches deployment height
        break
##########
```

Code Summary: This code block models the rocket's ascent using Euler's method. When the time from launch is under 5 seconds, the rocket experiences both gravity and thrust force. However, after that, the rocket only experiences gravity. Each time step's acceleration, velocity, and height are calculated from force, and a timestep is added after each loop. The ascent stops once the rocket's velocity becomes zero or negative, indicating that the rocket is at its apogee.

In [ ]:
```
######## RECOVERY

## SET INITIAL VARIABLES
t0 = 0      # starting time, s
```

```
dt = 0.05   # time step, s
height = height_launch[j] #apogee height, m
dheight = 100 #deployment height, m
rho = 1.225 #air density, kg/m^3
deploy_time = 0.3 #parachute 2 deployment period
collide_time = 0.1 #rocket collision time, s

t_max = 6000 #maximum time
n_steps = int((t_max - t0) / dt) + 1 # Total number of steps

## Arrays to save simulation information
time = np.zeros(n_steps)
position = np.zeros(n_steps)
velocity = np.zeros(n_steps)
drag = np.zeros(n_steps)
acc = np.zeros(n_steps)
instantforce = np.zeros(n_steps)

# Initial properties: Rocket
y1_initial = height # m
v1_initial = 0  # m/s
acc_initial = 0 #ms^2
force_initial = 0 #N
drag_initial = 0 #N
time[0] = t0 #s

r1 = 0.05 # m
position[0] = y1_initial #m
velocity[0] = v1_initial  #m/s
acc[0] = acc_initial #m/s^2
instantforce[0] = force_initial #Force on rocket at apogee, N
drag[0] = drag_initial # Drag force on rocket at apogee, N

gforce = mass * 9.81 # Force of gravity

# Fixed Initial properties: Parachute 1
cod1 = 0.7 #coefficient of drag
area1 = 5 #cross-sectional surface area, m^2

# VARIABLE INITIAL PROPERTIES: Parachute 2
cod2 = 1.5 #coefficient of drag
area2 = 130 #cross-sectional surface area, m^2
termvel = np.sqrt(2*mass*9.81/(rho*area2*cod2))  #terminal velocity, m/s^2
##########
```

Code Summary: This is the beginning of the rocket's recovery trajectory. This block defines all necessary variables, arrays, and constants. This includes Parachute 1's and Parachute 2's coefficients of drag and cross-sectional surface area. In this simulation, the initial properties of Parachute 2 are variable, but Parachute 1 is fixed.

In [ ]:
```
# PART 0.5: AFTER PARACHUTE 1 DEPLOYED

counter = 0
for i in range(1,n_steps):
    drag[i] = 0.5 * rho * cod1 * area1 * velocity[i-1]**2
    instantforce[i] = -gforce + drag[i]
    acc[i] = instantforce[i]/mass

    velocity[i] = velocity[i-1] - acc[i] * dt
    position[i] = position[i-1] - velocity[i] * dt

    time[i]  =  time[i-1] + dt
    counter = counter + 1
    if position[i] <= dheight: # If reaches deployment height
        break
##########
```

Code Summary: This code block models the rocket's descent after parachute 1 is deployed at the apogee using Earler's method and a for loop. The loop breaks when the rocket's position is less than or equal to dheight, signaling to the simulation that Parachute 2 should now be deployed.

In [ ]:
```
## PARACHUTE 2 DEPLOYMENT
mi = mass * velocity[counter-1]
```

```
mf = mass * termvel
dm = abs(mi - mf)
force = dm/deploy_time
```

Code Summary: This block models the rocket's parachute 2 deployment. The initial and final momentum is calculated through mi and mf, and the difference is calculated as dm. Force is then the change in momentum over deployment time, which was defined previously.

In [ ]:
```
# PARACHUTE 2 DEPLOYMENT FAILURE
if force > thresh_force:
    for i in range(len(velocity)):
        velocity[i] = -velocity[i]
    fig, axes = plt.subplots(1, 2)
    plt.subplots_adjust(wspace=0.5)
    fig.set_size_inches(9, 5)
    axes[0].plot(time[:counter+1],velocity[:counter+1], color = "brown")
    axes[0].set_xlim(0, 100)
    axes[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
    axes[0].set_xlabel("Time (s)")
    axes[0].set_ylabel("Velocity (m/s)")
    axes[0].text(time[counter], velocity[counter], 'Recovery Fail: \n Rocket Damaged', horizontalalignme
        bbox=dict(
            boxstyle='round,pad=0.5',
            fc='red',
            ec='black',
            lw=2,
            alpha=0.4
        ))

    axes[1].plot(time[:counter+1],position[:counter+1], color = "orange")
    axes[1].set_xlim(0,100)
    axes[1].set_title("Rocket Position vs. Time", fontweight = "bold")
    axes[1].set_xlabel("Time (s)")
    axes[1].set_ylabel("Position (m)")
    axes[1].text(time[counter], position[counter], 'Recovery Fail: \n Rocket Damaged', horizontalalignme
        bbox=dict(
            boxstyle='round,pad=0.5',
            fc='red',
            ec='black',
            lw=2,
            alpha=0.4
        ))

    fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
    plt.subplots_adjust(top=0.8)
    fig.text(0.5, 0.88, 'Status: Failed during Parachute 2 Deployment', ha='center', fontsize=14, color=

    print("Collision 1 Failed: Rocket experienced force greater than threshold during Parachute 2 deploy
    print(f"The threshold force was {thresh_force: 0.2f} N.")
    print(f"The force during parachute 2 deployment was {force: 0.2f} N.")


# CODE SUMMARY [1]

# PARACHUTE 2 DEPLOYMENT SUCCESS
else:
    print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployme
    dt = 1
    counter2 = counter

# PART II: CONSTANT DESCENT TO GROUND

    for i in range(counter,n_steps):
        position[i] = position[i-1] - termvel * dt
        counter2 = counter2 + 1
        time[i]  =  time[i-1] + dt
        velocity[i] = termvel
        if position[i] <= 0:
            velocity[i] = 0
            position[i] = 0
            break

# CODE SUMMARY: This code simulates the rocket's constant descent to ground given a successful Parachute
```

```
#Euler's method to continue the rocket's trajectory. A new counter, counter2, is made to extend counter

# PART III: GROUND COLLISION
    m11 = mass * velocity[counter2-2]
    force2 = m11/collide_time


#CODE SUMMARY: This code simulates the rocket's ground collision upon landing. Since we assume an entire
#is simply the momentum the rocket has immediately before its collision. The force is that momentum divi
#in previous code.

# GROUND COLLISION FAILURE
    if force2 >= thresh_force:
        for i in range(len(velocity)):
            velocity[i] = -velocity[i]

        print("Collision 2 Fail: Rocket experienced force greater than threshold during ground collision
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {force: 0.2f} N.")
        print(f"The force during the ground collision was {force2: 0.2f} N.")

        fig, axes2 = plt.subplots(1, 2)
        plt.subplots_adjust(wspace=0.5)
        fig.set_size_inches(13, 7)

        axes2[0].plot(time[:counter2],velocity[:counter2], color = "brown")
        axes2[0].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
        axes2[0].set_xlabel("Time (s)")
        axes2[0].set_ylabel("Velocity (m/s)")
        axes2[0].text(time[counter-1], (3*velocity[counter-1]-velocity[counter])/4, 'Success: Chute 2 \n
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[0].text(time[counter2-1], velocity[counter2-1], 'Fail: Damaged \n While Landing', horizont
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='red',
                ec='black',
                lw=2,
                alpha=0.4
            ))

        axes2[1].plot(time[:counter2],position[:counter2], color = "orange")
        axes2[1].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[1].set_title("Rocket Position vs. Time", fontweight = "bold")
        axes2[1].set_xlabel("Time (s)")
        axes2[1].set_ylabel("Position (m)")
        axes2[1].text(time[counter-1], (position[0] + position[counter-1])/2, 'Success: Chute \n 2 Deplo
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[1].text(time[counter2-1], position[counter2-1], 'Fail: Damaged \n While Landing', horizont
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc="red",
                ec='black',
                lw=2,
                alpha=0.4
            ))

        fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
        plt.subplots_adjust(top=0.72)
        fig.text(0.5, 0.89, 'Part I Status: Parachute 2 Deployment - Successs', ha='center', fontsize=14
        fig.text(0.5, 0.85, 'Part II Status: Rocket Reaching Earth - Success', ha='center', fontsize=14,
        fig.text(0.5, 0.81, 'Part III Status: Rocket Landing Safely - Fail', ha='center', fontsize=14, c
```

```
#CODE SUMMARY: This code executes the simulation's output if the ground collision force exceeds the thre
#Time graph up until the rocket lands, along with annotations displaying a successful Parachute 2 deploy
#parachute 2 deployment and ground collision is also printed.

# GROUND COLLISION SUCCESS, ROCKET RECOVERY SUCCESS
    else:
        for i in range(len(velocity)):
            velocity[i] = -velocity[i]
        print("Collision 2 Success: Rocket experienced force less than threshold during ground collision
        fig, axes2 = plt.subplots(1, 2)
        plt.subplots_adjust(wspace=0.5)
        fig.set_size_inches(12, 6)

        axes2[0].plot(time[:counter2],velocity[:counter2], color = "brown")
        axes2[0].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
        axes2[0].set_xlabel("Time (s)")
        axes2[0].set_ylabel("Velocity (m/s)")
        axes2[0].text(time[counter-1], (3*velocity[counter-1]-velocity[counter])/4, 'Success: Chute 2 \n
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[0].text(time[counter2-1], velocity[counter2-1], 'Success: Landed \n Safely', horizontalali
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))

        axes2[1].plot(time[:counter2],position[:counter2], color = "orange")
        axes2[1].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[1].set_title("Rocket Position vs. Time", fontweight = "bold")
        axes2[1].set_xlabel("Time (s)")
        axes2[1].set_ylabel("Position (m)")
        axes2[1].text(time[(counter-1)]/2,(position[0] + position[counter-1])/2, 'Success: Chute \n 2 De
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[1].text(time[counter2-1], position[counter2-1], 'Success: Landed \n Safely', horizontalali
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc="green",
                ec='black',
                lw=2,
                alpha=0.4
            ))

        fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
        plt.subplots_adjust(top=0.72)
        fig.text(0.5, 0.89, 'Part I Status: Parachute 2 Deployment - Successs', ha='center', fontsize=14
        fig.text(0.5, 0.85, 'Part II Status: Rocket Reaching Earth - Success', ha='center', fontsize=14,
        fig.text(0.5, 0.81, 'Part III Status: Rocket Landing Safely - Success!', ha='center', fontsize=1
        print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 depl
        print("Collision 2 Success: Rocket experienced less force than threshold during ground collision
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {force: 0.2f} N.")
        print(f"The force during the ground collision was {force2: 0.2f} N.")

trial1vel = velocity.copy()
trial1position = position.copy()
trial1time = time.copy()
trial1count = counter+1
```

```
#CODE SUMMARY: This code executes the simulation's output if the ground collision force is less than the
#recovery. It displays a Rocket Velocity vs. Time graph up until the rocket lands, along with annotation
#and ground collision. The forces during  parachute 2 deployment and ground collision are also printed.
```

```
Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployment
Collision 2 Success: Rocket experienced force less than threshold during ground collision

Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployment
Collision 2 Success: Rocket experienced less force than threshold during ground collision
The threshold force was  3500.00 N.
The force during parachute 2 deployment was  2712.48 N.
The force during the ground collision was  1678.46 N.
```

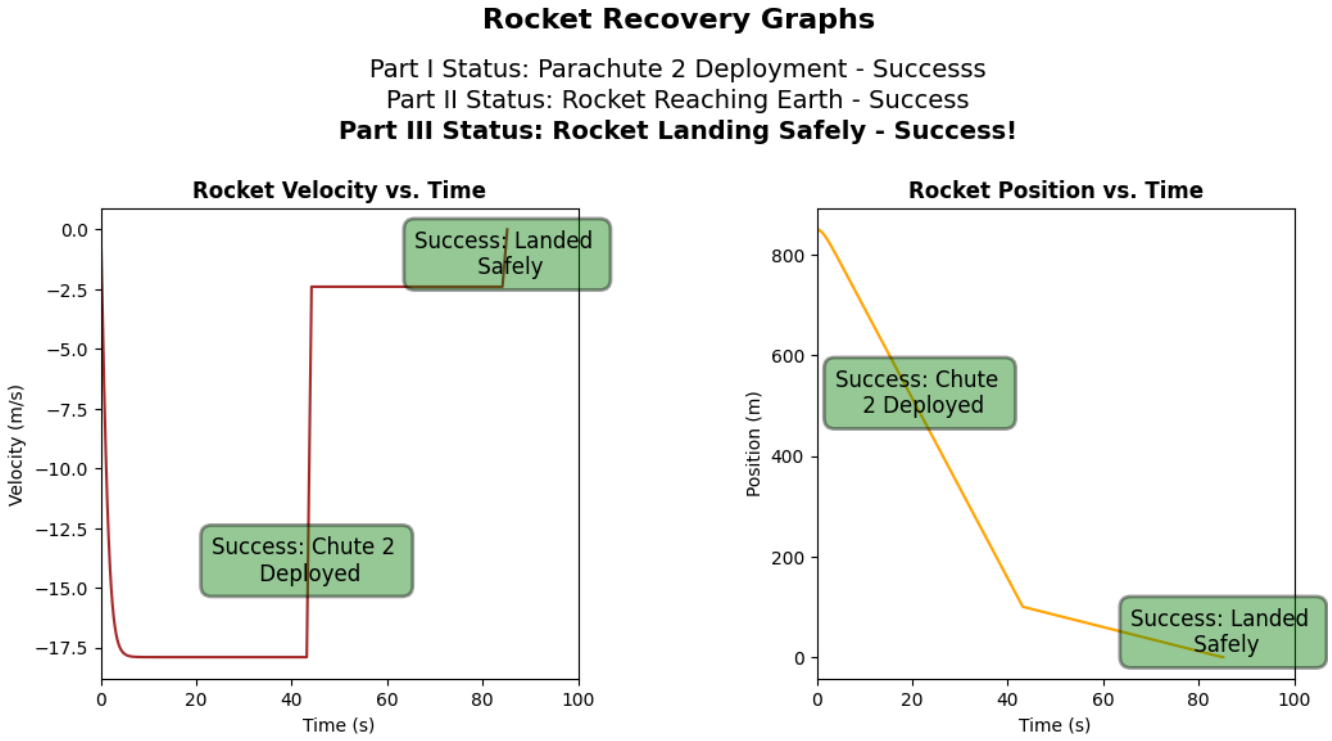


Figure A.2. A parachute deployment time of 0.3s yields a rocket recovery fail, where the rocket is damaged during parachute 2 deployment.

Trial 2: Parachute Deployment Time = $0.4s$

In [61]:

```
import numpy as np
import matplotlib.pyplot as plt

## USER INPUTS_THRESHOLD FORCE AND THRUST FORCE
thrust_force = 1000
Thresh_force = 3500

##### LAUNCHING

## CREATING EMPTY LISTS AND INITIAL VALUES FOR EACH VARIABLE
mass = 70 # Rocket kg
force_launch = []

height_launch = []
height_launch.append(0)

acceleration_launch = []

velocity_launch = []
velocity_launch.append(0)

t_launch = []
t_launch.append(0)
dt_launch = 0.1


##########
```

Code Summary: This code block creates empty lists and initial values for the needed variables for the simulation, including force, height, acceleration, velocity, and time. The rocket's thrust force and threshold force of 3500 are also defined.

In [62]:
```
## PART 0: ASCENT TO APOGEE

#### Modeling ascent using Euler's Method with while loop
j = 0
while True:
    if t_launch[j] <= 5:
        force_launch.append(thrust_force - mass * 9.8)
    else:
        force_launch.append(-mass*9.8)
    acc = force_launch[j]/mass
    acceleration_launch.append(acc)
    velocity_launch.append(velocity_launch[j] + acceleration_launch[j])
    height_launch.append(height_launch[j] + velocity_launch[j] * dt_launch)
    t_launch.append(t_launch[j] + dt_launch)

    j = j + 1
    if velocity_launch[j] <= 0: # If reaches deployment height
        break
##########
```

Code Summary: This code block models the rocket's ascent using Euler's method. When the time from launch is under 5 seconds, the rocket experiences both gravity and thrust force. However, after that, the rocket only experiences gravity. Each time step's acceleration, velocity, and height are calculated from force, and a timestep is added after each loop. The ascent stops once the rocket's velocity becomes zero or negative, indicating that the rocket is at its apogee.

In [63]:
```
######## RECOVERY

## SET INITIAL VARIABLES
t0 = 0      # starting time, s
dt = 0.05   # time step, s
height = height_launch[j] #apogee height, m
dheight = 100 #deployment height, m
rho = 1.225 #air density, kg/m^3
deploy_time = 0.4 #parachute 2 deployment period
collide_time = 0.1 #rocket collision time, s

t_max = 6000 #maximum time
n_steps = int((t_max - t0) / dt) + 1 # Total number of steps

## Arrays to save simulation information
time = np.zeros(n_steps)
position = np.zeros(n_steps)
velocity = np.zeros(n_steps)
drag = np.zeros(n_steps)
acc = np.zeros(n_steps)
instantforce = np.zeros(n_steps)

# Initial properties: Rocket
y1_initial = height # m
v1_initial = 0  # m/s
acc_initial = 0 #ms^2
force_initial = 0 #N
drag_initial = 0 #N
time[0] = t0 #s

r1 = 0.05 # m
position[0] = y1_initial #m
velocity[0] = v1_initial  #m/s
acc[0] = acc_initial #m/s^2
instantforce[0] = force_initial #Force on rocket at apogee, N
drag[0] = drag_initial # Drag force on rocket at apogee, N

gforce = mass * 9.81 # Force of gravity

# Fixed Initial properties: Parachute 1
cod1 = 0.7 #coefficient of drag
area1 = 5 #cross-sectional surface area, m^2

# VARIABLE INITIAL PROPERTIES: Parachute 2
```

```
cod2 = 1.5 #coefficient of drag
area2 = 130 #cross-sectional surface area, m^2
termvel = np.sqrt(2*mass*9.81/(rho*area2*cod2))  #terminal velocity, m/s^2
##########
```

Code Summary: This is the beginning of the rocket's recovery trajectory. This block defines all necessary variables, arrays, and constants. This includes Parachute 1's and Parachute 2's coefficients of drag and cross-sectional surface area. In this simulation, the initial properties of Parachute 2 are variable, but Parachute 1 is fixed.

In [64]:
```
# PART 0.5: AFTER PARACHUTE 1 DEPLOYED

counter = 0
for i in range(1,n_steps):
    drag[i] = 0.5 * rho * cod1 * area1 * velocity[i-1]**2
    instantforce[i] = -gforce + drag[i]
    acc[i] = instantforce[i]/mass

    velocity[i] = velocity[i-1] - acc[i] * dt
    position[i] = position[i-1] - velocity[i] * dt

    time[i]  =  time[i-1] + dt
    counter = counter + 1
    if position[i] <= dheight: # If reaches deployment height
        break
##########
```

Code Summary: This code block models the rocket's descent after parachute 1 is deployed at the apogee using Earler's method and a for loop. The loop breaks when the rocket's position is less than or equal to dheight, signaling to the simulation that Parachute 2 should now be deployed.

In [65]:
```
## PARACHUTE 2 DEPLOYMENT
mi = mass * velocity[counter-1]
mf = mass * termvel
dm = abs(mi - mf)
force = dm/deploy_time
```

Code Summary: This block models the rocket's parachute 2 deployment. The initial and final momentum is calculated through mi and mf, and the difference is calculated as dm. Force is then the change in momentum over deployment time, which was defined previously.

In [66]:
```
# PARACHUTE 2 DEPLOYMENT FAILURE
if force > thresh_force:
    for i in range(len(velocity)):
        velocity[i] = -velocity[i]
    fig, axes = plt.subplots(1, 2)
    plt.subplots_adjust(wspace=0.5)
    fig.set_size_inches(9, 5)
    axes[0].plot(time[:counter+1],velocity[:counter+1], color = "brown")
    axes[0].set_xlim(0, 100)
    axes[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
    axes[0].set_xlabel("Time (s)")
    axes[0].set_ylabel("Velocity (m/s)")
    axes[0].text(time[counter], velocity[counter], 'Recovery Fail: \n Rocket Damaged', horizontalalignme
        bbox=dict(
            boxstyle='round,pad=0.5',
            fc='red',
            ec='black',
            lw=2,
            alpha=0.4
        ))

    axes[1].plot(time[:counter+1],position[:counter+1], color = "orange")
    axes[1].set_xlim(0,100)
    axes[1].set_title("Rocket Position vs. Time", fontweight = "bold")
    axes[1].set_xlabel("Time (s)")
    axes[1].set_ylabel("Position (m)")
    axes[1].text(time[counter], position[counter], 'Recovery Fail: \n Rocket Damaged', horizontalalignme
        bbox=dict(
            boxstyle='round,pad=0.5',
            fc='red',
            ec='black',
```

```
            lw=2,
            alpha=0.4
        ))

    fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
    plt.subplots_adjust(top=0.8)
    fig.text(0.5, 0.88, 'Status: Failed during Parachute 2 Deployment', ha='center', fontsize=14, color=

    print("Collision 1 Failed: Rocket experienced force greater than threshold during Parachute 2 deploy
    print(f"The threshold force was {thresh_force: 0.2f} N.")
    print(f"The force during parachute 2 deployment was {force: 0.2f} N.")


# CODE SUMMARY [1]

# PARACHUTE 2 DEPLOYMENT SUCCESS
else:
    print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployme
    dt = 1
    counter2 = counter

# PART II: CONSTANT DESCENT TO GROUND

    for i in range(counter,n_steps):
        position[i] = position[i-1] - termvel * dt
        counter2 = counter2 + 1
        time[i]  =  time[i-1] + dt
        velocity[i] = termvel
        if position[i] <= 0:
            velocity[i] = 0
            position[i] = 0
            break

# CODE SUMMARY: This code simulates the rocket's constant descent to ground given a successful Parachute
#Euler's method to continue the rocket's trajectory. A new counter, counter2, is made to extend counter

# PART III: GROUND COLLISION
    m11 = mass * velocity[counter2-2]
    force2 = m11/collide_time


#CODE SUMMARY: This code simulates the rocket's ground collision upon landing. Since we assume an entire
#is simply the momentum the rocket has immediately before its collision. The force is that momentum divi
#in previous code.

# GROUND COLLISION FAILURE
    if force2 >= thresh_force:
        for i in range(len(velocity)):
            velocity[i] = -velocity[i]

        print("Collision 2 Fail: Rocket experienced force greater than threshold during ground collision
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {force: 0.2f} N.")
        print(f"The force during the ground collision was {force2: 0.2f} N.")

        fig, axes2 = plt.subplots(1, 2)
        plt.subplots_adjust(wspace=0.5)
        fig.set_size_inches(13, 7)

        axes2[0].plot(time[:counter2],velocity[:counter2], color = "brown")
        axes2[0].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
        axes2[0].set_xlabel("Time (s)")
        axes2[0].set_ylabel("Velocity (m/s)")
        axes2[0].text(time[counter-1], (3*velocity[counter-1]-velocity[counter])/4, 'Success: Chute 2 \n
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[0].text(time[counter2-1], velocity[counter2-1], 'Fail: Damaged \n While Landing', horizont
            bbox=dict(
```

```
                boxstyle='round,pad=0.5',
                fc='red',
                ec='black',
                lw=2,
                alpha=0.4
            ))

        axes2[1].plot(time[:counter2],position[:counter2], color = "orange")
        axes2[1].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[1].set_title("Rocket Position vs. Time", fontweight = "bold")
        axes2[1].set_xlabel("Time (s)")
        axes2[1].set_ylabel("Position (m)")
        axes2[1].text(time[counter-1], (position[0] + position[counter-1])/2, 'Success: Chute \n 2 Deplo
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[1].text(time[counter2-1], position[counter2-1], 'Fail: Damaged \n While Landing', horizont
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc="red",
                ec='black',
                lw=2,
                alpha=0.4
            ))

        fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
        plt.subplots_adjust(top=0.72)
        fig.text(0.5, 0.89, 'Part I Status: Parachute 2 Deployment - Successs', ha='center', fontsize=14
        fig.text(0.5, 0.85, 'Part II Status: Rocket Reaching Earth - Success', ha='center', fontsize=14,
        fig.text(0.5, 0.81, 'Part III Status: Rocket Landing Safely - Fail', ha='center', fontsize=14, c

#CODE SUMMARY: This code executes the simulation's output if the ground collision force exceeds the thre
#Time graph up until the rocket lands, along with annotations displaying a successful Parachute 2 deploy
#parachute 2 deployment and ground collision is also printed.

# GROUND COLLISION SUCCESS, ROCKET RECOVERY SUCCESS
    else:
        for i in range(len(velocity)):
            velocity[i] = -velocity[i]
        print("Collision 2 Success: Rocket experienced force less than threshold during ground collision
        fig, axes2 = plt.subplots(1, 2)
        plt.subplots_adjust(wspace=0.5)
        fig.set_size_inches(12, 6)

        axes2[0].plot(time[:counter2],velocity[:counter2], color = "brown")
        axes2[0].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[0].set_title("Rocket Velocity vs. Time", fontweight = "bold")
        axes2[0].set_xlabel("Time (s)")
        axes2[0].set_ylabel("Velocity (m/s)")
        axes2[0].text(time[counter-1], (3*velocity[counter-1]-velocity[counter])/4, 'Success: Chute 2 \n
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[0].text(time[counter2-1], velocity[counter2-1], 'Success: Landed \n Safely', horizontalali
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))

        axes2[1].plot(time[:counter2],position[:counter2], color = "orange")
        axes2[1].set_xlim(0,(time[counter2-1] // 50)*50 + 50)
        axes2[1].set_title("Rocket Position vs. Time", fontweight = "bold")
        axes2[1].set_xlabel("Time (s)")
```

```
        axes2[1].set_ylabel("Position (m)")
        axes2[1].text(time[(counter-1)]/2,(position[0] + position[counter-1])/2, 'Success: Chute \n 2 De
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc='green',
                ec='black',
                lw=2,
                alpha=0.4
            ))
        axes2[1].text(time[counter2-1], position[counter2-1], 'Success: Landed \n Safely', horizontalali
            bbox=dict(
                boxstyle='round,pad=0.5',
                fc="green",
                ec='black',
                lw=2,
                alpha=0.4
            ))

        fig.suptitle('Rocket Recovery Graphs', fontsize=16, fontweight='bold')
        plt.subplots_adjust(top=0.72)
        fig.text(0.5, 0.89, 'Part I Status: Parachute 2 Deployment - Successs', ha='center', fontsize=14
        fig.text(0.5, 0.85, 'Part II Status: Rocket Reaching Earth - Success', ha='center', fontsize=14,
        fig.text(0.5, 0.81, 'Part III Status: Rocket Landing Safely - Success!', ha='center', fontsize=1
        print("Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 depl
        print("Collision 2 Success: Rocket experienced less force than threshold during ground collision
        print(f"The threshold force was {thresh_force: 0.2f} N.")
        print(f"The force during parachute 2 deployment was {force: 0.2f} N.")
        print(f"The force during the ground collision was {force2: 0.2f} N.")

trial1vel = velocity.copy()
trial1position = position.copy()
trial1time = time.copy()
trial1count = counter+1

#CODE SUMMARY: This code executes the simulation's output if the ground collision force is less than the
#recovery. It displays a Rocket Velocity vs. Time graph up until the rocket lands, along with annotation
#and ground collision. The forces during  parachute 2 deployment and ground collision are also printed.
```

```
Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployment
Collision 2 Success: Rocket experienced force less than threshold during ground collision

Collision 1 Success: Rocket experienced force less than threshold during Parachute 2 deployment
Collision 2 Success: Rocket experienced less force than threshold during ground collision
The threshold force was  3500.00 N.
The force during parachute 2 deployment was  2712.48 N.
The force during the ground collision was  1678.46 N.
```

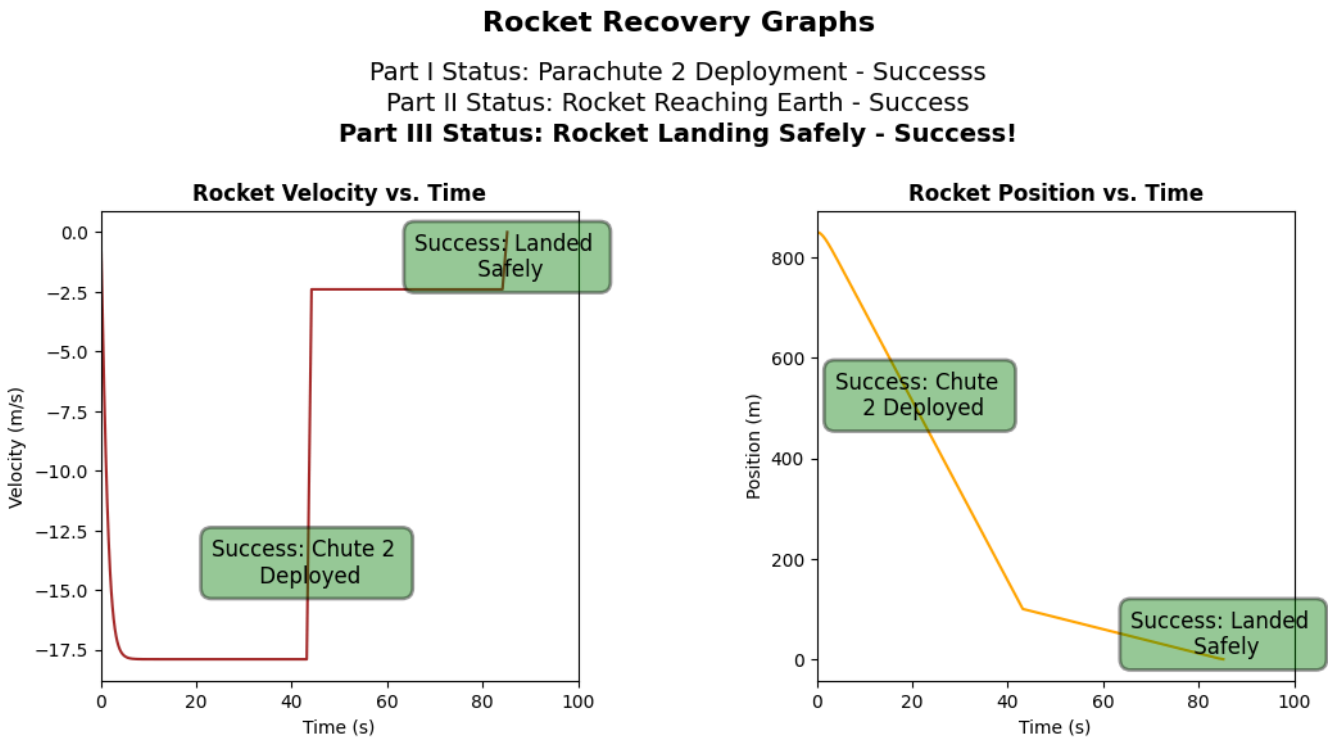


Figure A.3. A parachute deployment time of 0.4s yields a successful parachute landing.

Analysis

From **Figure A.2** and **Figure A.3**, we see that a change in 0.1s in parachute 2 deployment time is enough time for the simulation to calculate a ~900N difference in parachute 2 deployment force, resulting in a switch from a failed to a successful rocket recovery. This analysis indicates that the simulation is highly sensitive.